HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

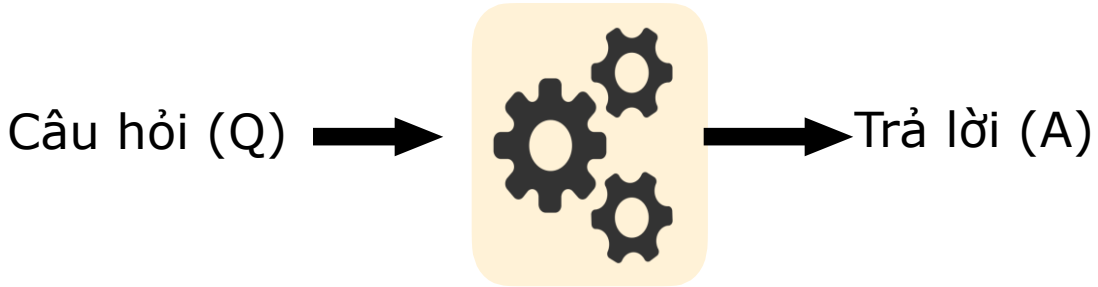
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

Xây dựng hệ thống hỏi đáp sử dụng tìm kiếm và RAG

Hệ thống hỏi đáp

- Mục tiêu: Xây dựng các hệ thống tự động trả lời các câu hỏi của con người bằng ngôn ngữ tự nhiên



- Nguồn thông tin:
  - Đoạn văn bản, các tài liệu trên web, cơ sở tri thức, cơ sở dữ liệu, tập các câu hỏi đáp có sẵn
- Các dạng câu hỏi: trả về giá trị/trả về không phải là giá trị, miền đóng/miền mở, đơn giản/phức tạp, …
- Các dạng câu trả lời: một vài từ, một đoạn, danh sách, có/không, …

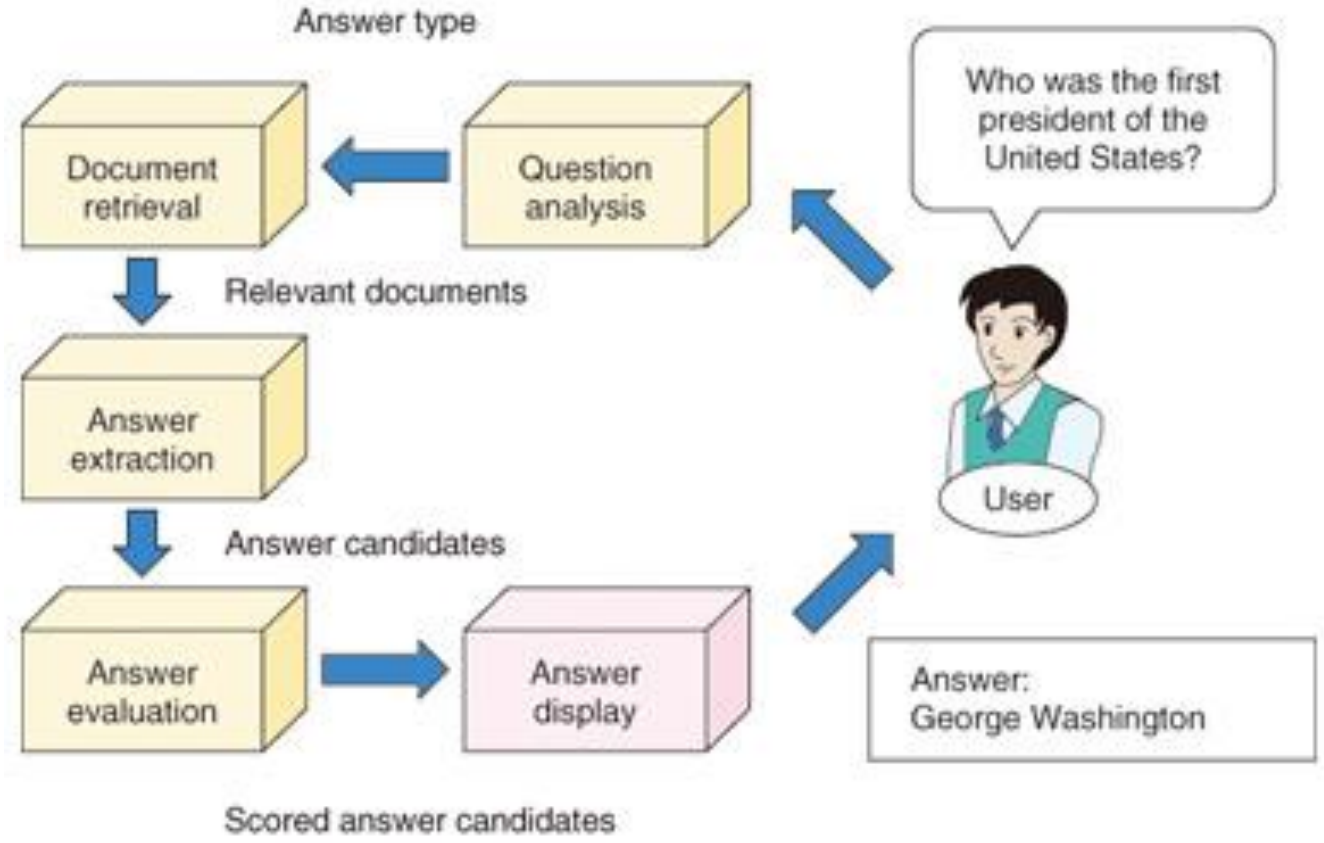
Các dạng câu hỏi

- Factoid queries: WH questions like when, who, where.
- Yes/ No queries: Is Berlin capital of Germany?
- Definition queries: what is leukemia?
- Cause/consequence queries: How, Why, What. what are the consequences of the Iraq war?
- Procedural queries: which are the steps for getting a Master degree?
- Comparative queries: what are the differences between the model A and B?
- Queries with examples: list of hard disks similar to hard disk X.
- Queries about opinion: What is the opinion of the majority of Americans about the Iraq war?

Các cách tiếp cận

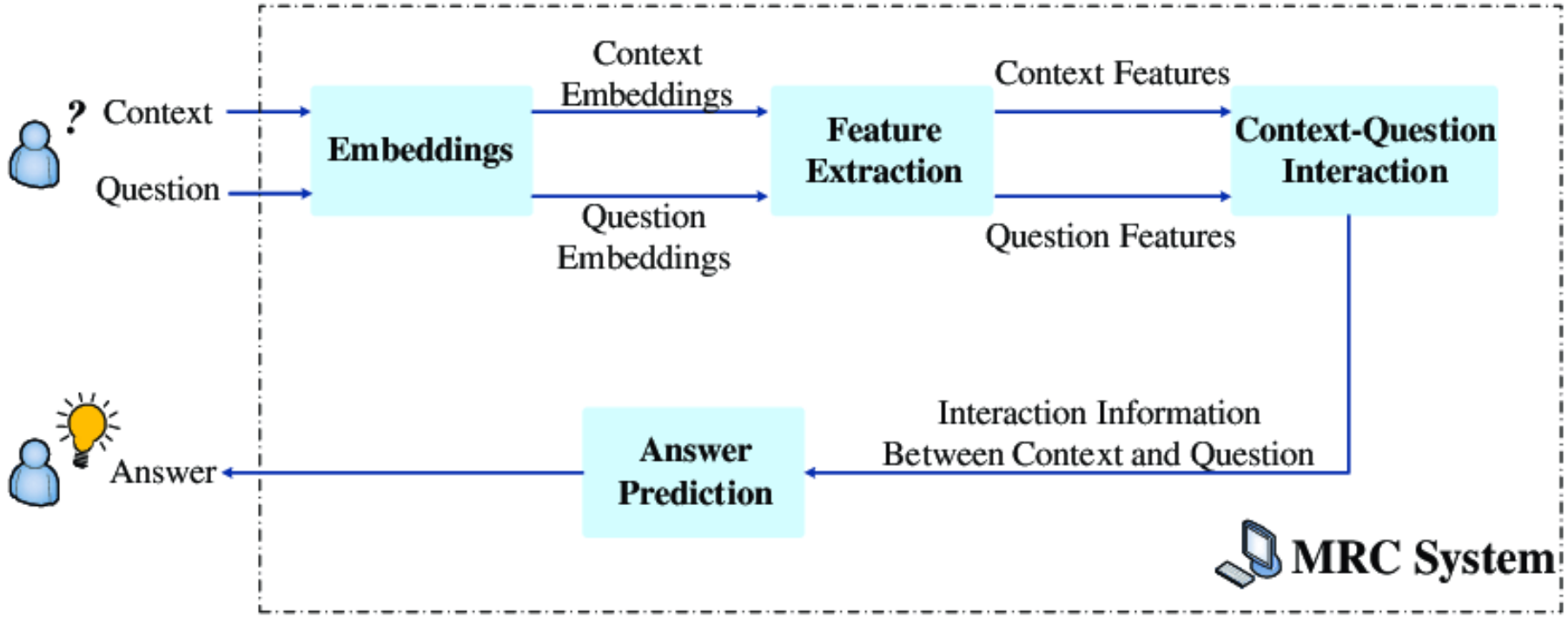
- Có bộ dữ liệu QA cho trước
  - Đo độ tương đồng câu, lấy câu trả lời của câu hỏi tương đồng nhất.
    - VD: AskJeeves
  - Huấn luyện sử dụng học sâu để dự đoán câu trả lời
- Không có bộ dữ liệu QA, có CSDL hoặc CSTT
  - Phân tích câu hỏi (phân tích ngữ nghĩa sâu, so khớp mẫu,…), tìm câu trả lời (tra cứu CSDL, so khớp mẫu, suy diễn, …)
    - VD: TextMap, AskMSR, LCC, …
  - Tìm kiếm các tài liệu liên quan, tìm câu trả lời từ tài liệu liên quan

Hệ thống hỏi đáp tự động



Hỏi đáp văn bản (Machine Reading Comprehension)

- Trả lời câu hỏi dựa trên một đoạn văn bản cụ thể.



Hỏi đáp văn bản (Machine Reading Comprehension)

- SQuAD được sử dụng rộng rãi trong việc xây dựng các hệ thống hỏi đáp.
  - 100k mẫu đã gán nhãn (đoạn văn bản, câu hỏi, câu trả lời)
  - Các đoạn được lấy từ English Wikipedia, 100~150 từ.
  - Câu hỏi do cộng đồng tạo ra
  - Mỗi câu trả lời là một xâu ngắn trong đoạn văn bản.

Hạn chế: không phải tất cả các câu hỏi đều có câu trả lời kiểu này!

  - Mỗi câu hỏi có 3 câu trả lời mẫu, vì thường có nhiều cách trả lời

In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called "showers".

What causes precipitation to fall?
gravity

What is another main form of precipitation besides drizzle, rain, snow, sleet and hail?
graupel

Where do water droplets collide with ice crystals to form precipitation?
within a cloud

Hỏi đáp miền mở (Open-Domain QA)

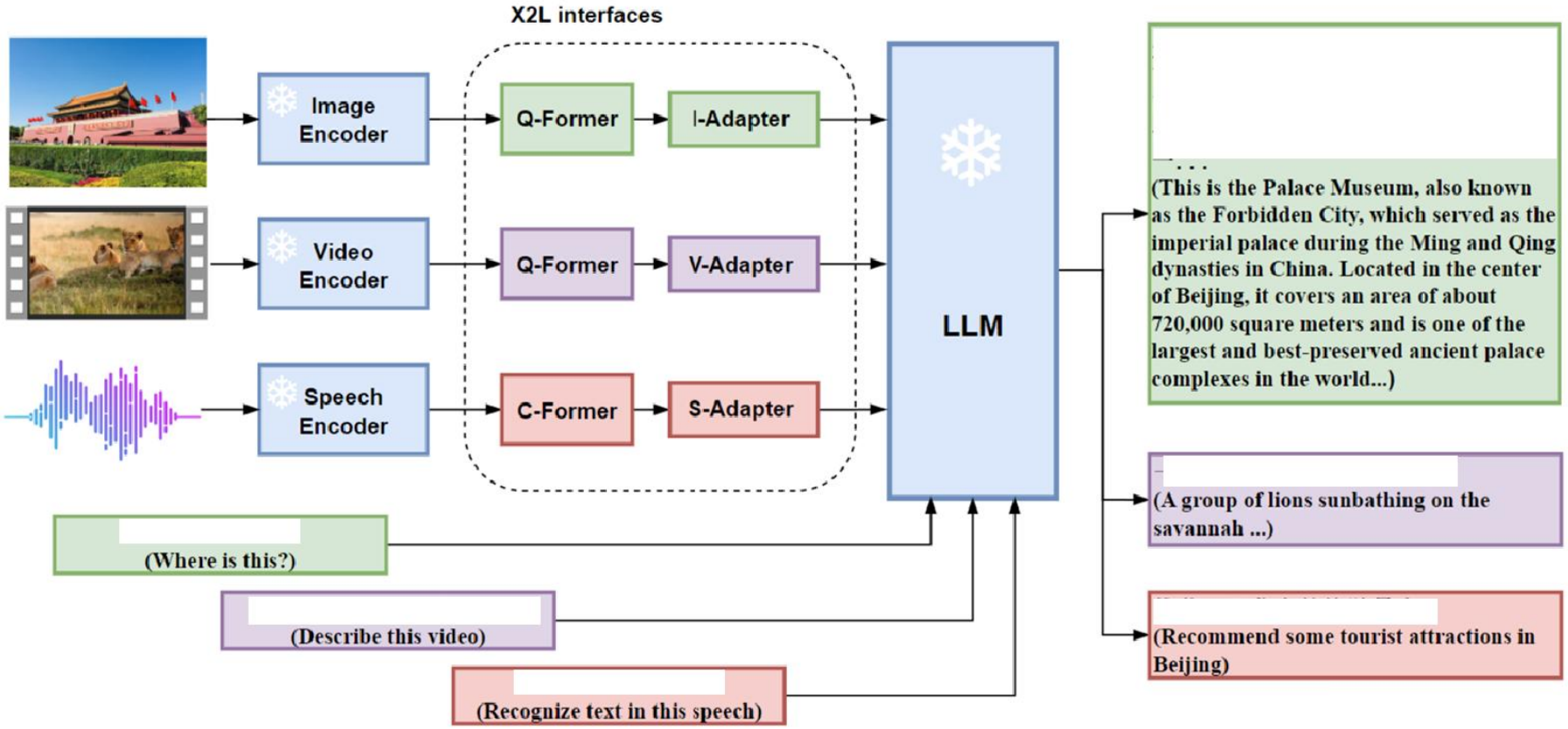
- Không giới hạn văn bản đầu vào – hệ thống phải tìm câu trả lời từ một kho văn bản lớn (Wikipedia, CSDL véc-tơ...).

  - Không cho trước 1 đoạn nội dung
  - Tìm kiếm trên tập lớn các tài liệu (như Wikipedia). Cần trả về câu trả lời cho bất kỳ miền dữ liệu nào
  - Thách thức hơn nhưng thực tế hơn!
  - https://huggingface.co/datasets/stanford-oval/wikipedia_20240801_10-languages_bge-m3_qdrant_index

Multi-modal QA

- Hỏi đáp trên sự kết hợp của nhiều loại dữ liệu đầu vào.
  - Ví dụ: một câu hỏi liên quan đến một biểu đồ, một hình ảnh y tế kèm chú thích, hay video có lời thoại
- **CLIP (OpenAI)**: Hiểu mối quan hệ giữa văn bản và hình ảnh.
- **VisualBERT, LXMERT, ViLT**: Các mô hình BERT mở rộng cho ảnh và văn bản.
- **GPT-4 (với khả năng multimodal)**: Có thể xử lý cả ảnh và văn bản để tạo câu trả lời.
- **Flamingo, Kosmos-1 (Microsoft), Gemini (Google)**: Các mô hình tiên tiến cho QA đa phương thức.

Multi-modal QA



Ôn tập kiến thức: Thực hành

- Một số cách tiếp cận cơ bản:
  - Keyword search, feature search
  - Cross-encoder
  - Bi-encoder

BM25 (Best Matching)

- Ước lượng mức độ liên quan của các tài liệu đối với một truy vấn, sử dụng biểu diễn tài liệu tương tự biểu diễn TF-IDF
- Ưu điểm
  - Đơn giản, dễ hiểu, hiệu quả
  - Sử dụng chuẩn hóa độ dài tài liệu
  - Tốc độ nhanh
- Hạn chế
  - Không xem xét ngữ nghĩa và ngữ cảnh
  - Giả định độc lập thống kê giữa các từ truy vấn

BM25 (Best Matching)

Cho một câu truy vấn Q chứa các từ $q_1, q_2, \ldots, q_n$, điểm BM25 của một tài liệu D là:

$$\text{score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{\text{TF}(q_i, D) \cdot (k_1 + 1)}{\text{TF}(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{avgdl})}$$

trong đó $\text{TF}(q_i, D)$ là số lần từ q_i xuất hiện trong tài liệu D, $|D|$ là độ dài của tài liệu D tính bằng số từ, $avgdl$ là độ dài trung bình của các tài liệu, k_1 và b là các siêu tham số, $\text{IDF}(q_i)$ là trọng số IDF của từ truy vấn q_i, thường được tính bởi:

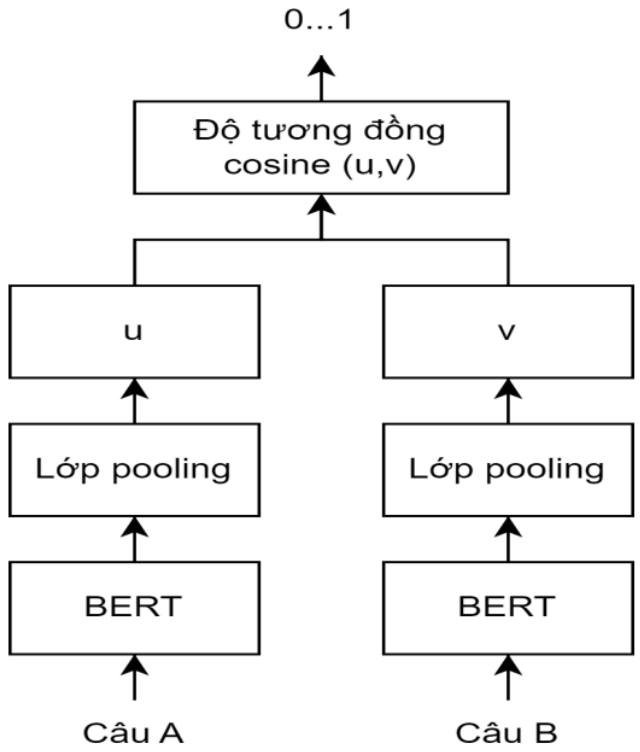
$$\text{IDF}(q_i) = \ln(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1)$$

trong đó N là tổng số tài liệu và $n(q_i)$ là số tài liệu chứa từ q_i.

Do các tài liệu dài thường có nhiều lần xuất hiện của một từ, siêu tham số b có tác dụng chuẩn hóa độ dài tài liệu với $\frac{|D|}{avgdl}$. Siêu tham số k_1 giảm thiểu ảnh hưởng của các từ có tần suất quá cao do chúng thường mang ít thông tin quan trọng.

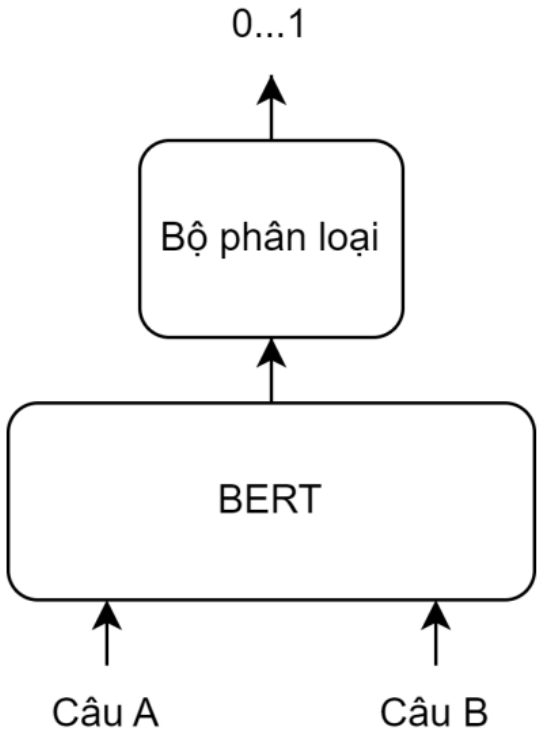
Bi-encoder

- **Bi-encoder** ánh xạ độc lập câu hỏi và văn bản vào không gian vector để so sánh độ tương đồng để tìm nội dung liên quan nhất.
- Là phương pháp phổ biến trong các hệ thống hỏi đáp hiện đại (QA), tìm kiếm ngữ nghĩa (semantic search), và vector search.
- Ưu điểm
  - Truy xuất dựa trên độ tương đồng ngữ nghĩa
  - Tốc độ truy xuất nhanh
  - Các vector nhúng có thể được tính trước



Cross-encoder

- Cross-encoder: **ghép hai chuỗi vào chung một mô hình**, và mô hình xử lý toàn bộ tương tác giữa chúng một cách trực tiếp
- So với kiến trúc bi-encoder
  - Thường độ chính xác cao hơn
  - Tốc độ chậm hơn



BGE-M3

- Three challenges:
  - Most of the embedding models are tailored only for English, leaving few viable options for the other languages.
  - The existing embedding models are usually trained for one single retrieval functionality. However, typical IR systems call for the compound workflow of multiple retrieval methods.
  - Most of the embedding models can only support short inputs
- Contribution
  - Multi-linguality
  - Multi-functionality: Dense retrieval, lexical (sparse) retrieval, and multi-vector retrieval
  - Multi-granularity: Input granularities, spanning from short inputs like sentences and passages, to long documents of up to 8,192 input tokens.
  - Propose a novel training framework of self-knowledge distillation and efficient batching strategy

Chen, J., Xiao, S., Zhang, P., Luo, K., Lian, D., & Liu, Z. (2024). Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*.

Architecture

- Adopt a further pre-trained XLM-RoBERTa12 as the foundational model
- Extend the max position to 8192


```
{
  "_name_or_path": "",
  "architectures": [
    "XLMRobertaModel"
  ],
  "attention_probs_dropout_prob": 0.1,
  "bos_token_id": 0,
  "classifier_dropout": null,
  "eos_token_id": 2,
  "hidden_act": "gelu",
  "hidden_dropout_prob": 0.1,
  "hidden_size": 1024,
  "initializer_range": 0.02,
  "intermediate_size": 4096,
  "layer_norm_eps": 1e-05,
  "max position embeddings": 8194,
  "model_type": "xlm-roberta",
  "num_attention_heads": 16,
  "num_hidden_layers": 24,
  "output_past": true,
  "pad_token_id": 1,
  "position_embedding_type": "absolute",
  "torch_dtype": "float32",
  "transformers_version": "4.33.0",
  "type_vocab_size": 1,
  "use_cache": true,
  "vocab_size": 250002
}
```

Datasets: Unsupervised and Supervised datasets

- Generate synthetic data to mitigate the shortage of long document retrieval tasks and introduce extra multi-lingual fine-tuning data (denoted as MultiLongDoc)
- Sample lengthy articles from Wikipedia, Wudao and mC4 datasets and randomly choose paragraphs from them.
- Use GPT3.5 to generate questions based on these paragraphs.
- The generated question and the sampled article constitute a new text pair to the fine-tuning data

Data Source	Language	Size
Unsupervised Data		
MTP	EN, ZH	291.1M
S2ORC, Wikipeida	EN	48.3M
xP3, mC4, CC-News	Multi-Lingual	488.4M
NLLB, CCMatrix	Cross-Lingual	391.3M
CodeSearchNet	Text-Code	344.1K
Total	–	1.2B
Fine-tuning Data		
MS MARCO, HotpotQA, NQ, NLI, etc.	EN	1.1M
DuReader, T^2-Ranking, NLI-zh, etc.	ZH	386.6K
MIRACL, Mr.TyDi	Multi-Lingual	88.9K
MultiLongDoc	Multi-Lingual	41.4K

Table 1: Specification of training data.

Training process:

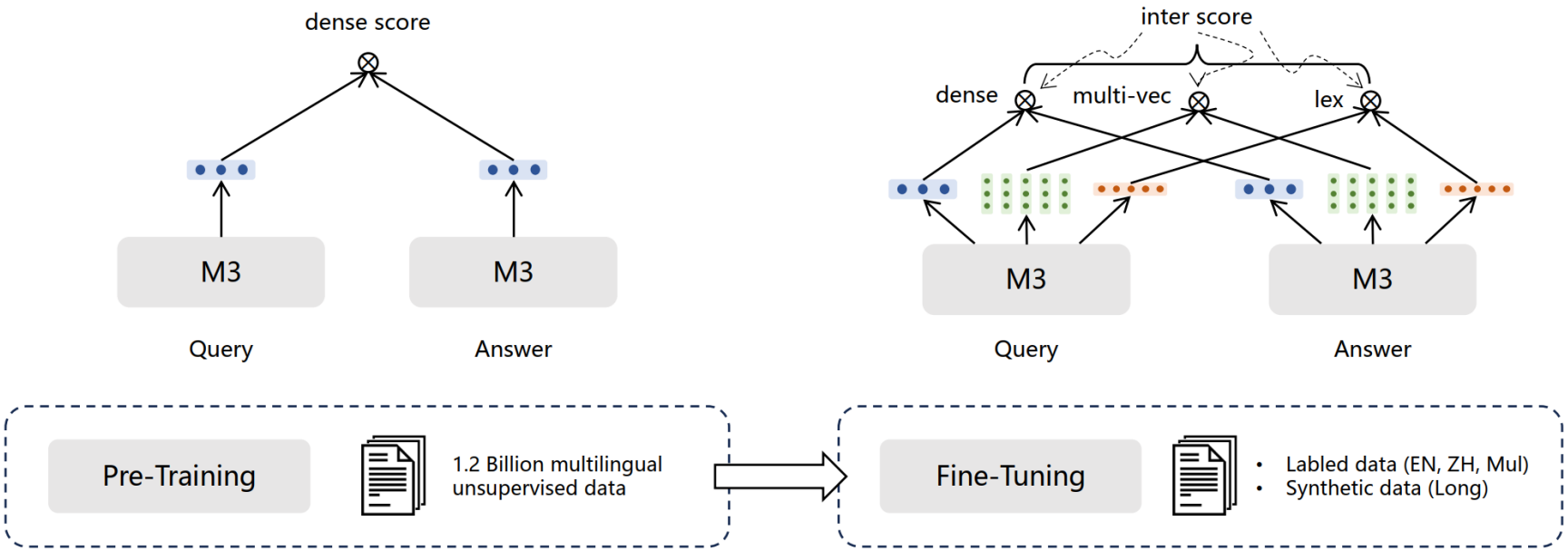
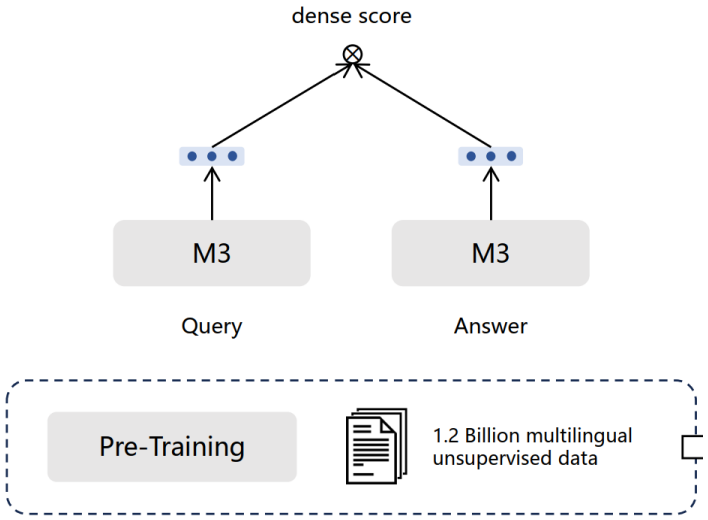


Figure 2: Multi-stage training process of M3-Embedding with self-knowledge distillation.

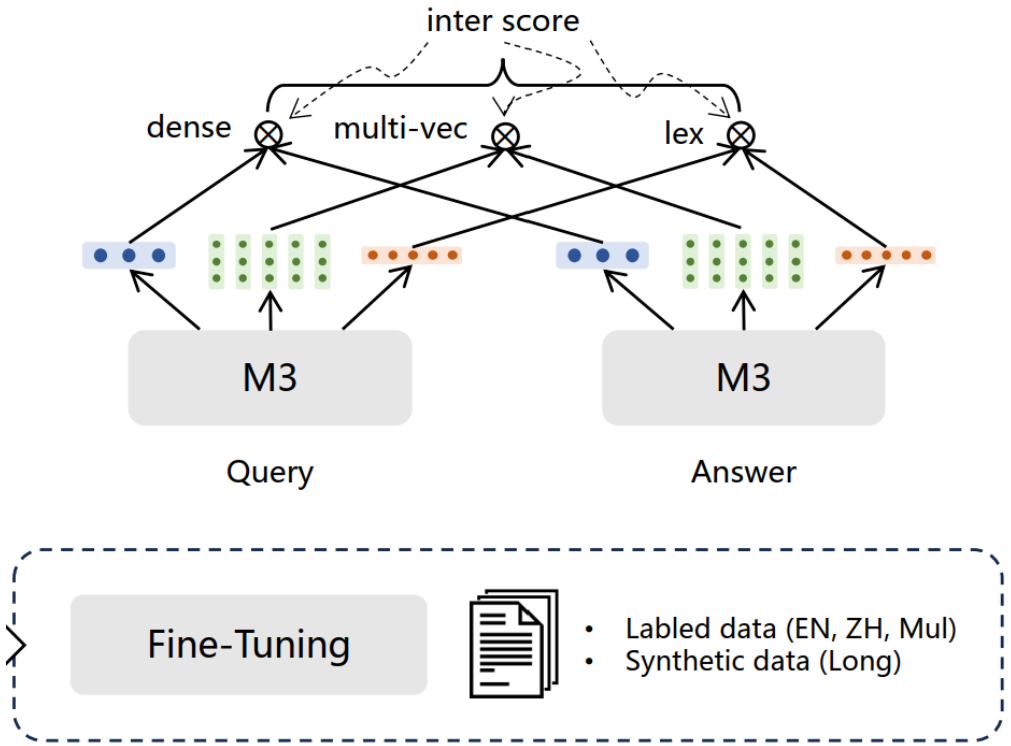
Phase 1: Pre-training

- The text encoder is pre-trained with the massive unsupervised data, where only the dense retrieval is trained in the basic form of contrastive learning



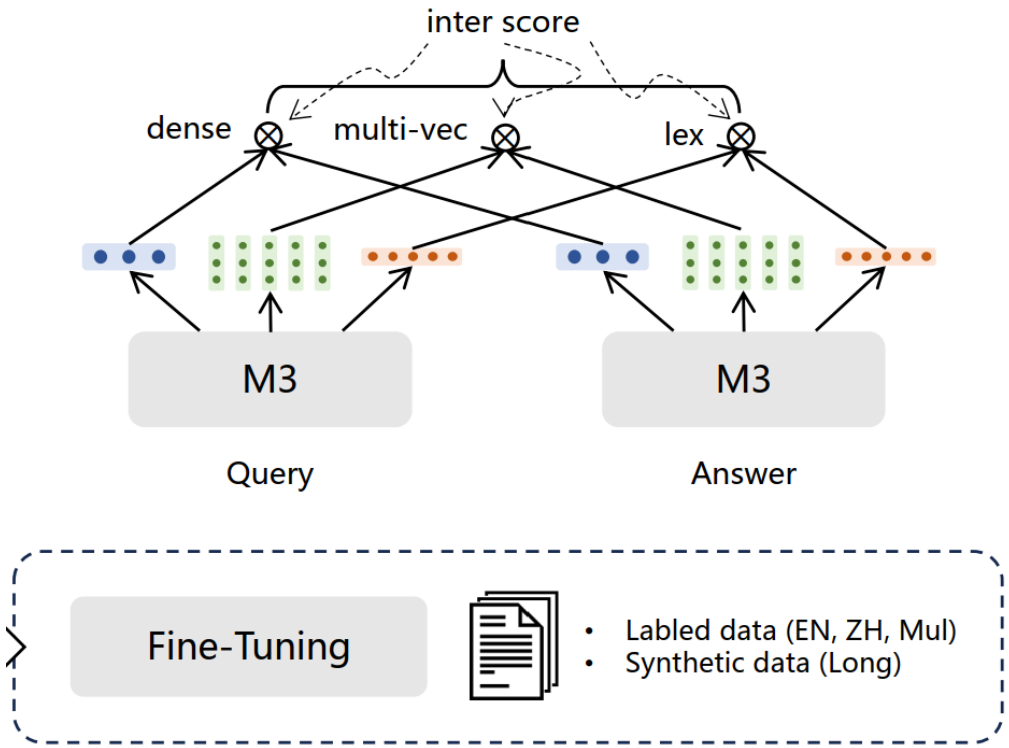
Phase 2: Finetuning

• **Dense retrieval**. The input query q is transformed into the hidden states $\mathbf{H_q}$ based on a text encoder. We use the normalized hidden state of the special token "[CLS]" for the representation of the query: $e_q = norm(\mathbf{H_q}[0])$. Similarly, we can get the embedding of passage p as $e_p = norm(\mathbf{H_p}[0])$. Thus, the relevance score between query and passage is measured by the inner product between the two embeddings e_q and e_p: $s_{dense} \leftarrow \langle e_p, e_q \rangle$.



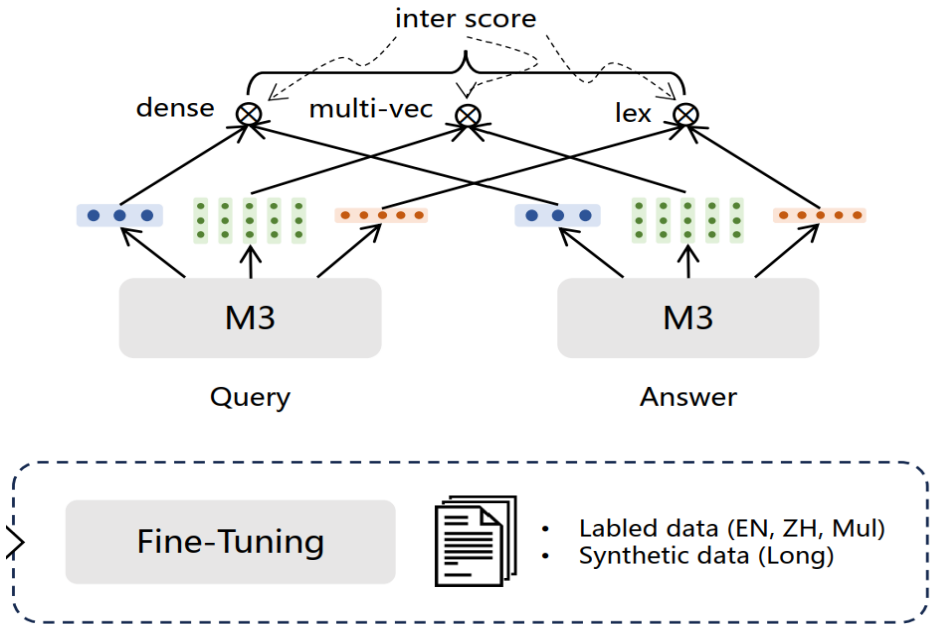
Phase 2: Finetuning

• **Lexical Retrieval**. The output embeddings are also used to estimate the importance of each term to facilitate lexical retrieval. For each term t within the query (a term is corresponding to a token in our work), the term weight is computed as $w_{q_t} \leftarrow \text{Relu}(\mathbf{W}_{lex}^T \mathbf{H}_\mathbf{q}[i]))$, where $\mathbf{W}_{lex} \in \mathcal{R}^{d \times 1}$ is the matrix mapping the hidden state to a float number. If a term t appears multiple times in the query, we only retain its max weight. We use the same way to compute the weight of each term in the passage. Based on the estimation term weights, the relevance score between query and passage is computed by the joint importance of the co-existed terms (denoted as $q \cap p$) within the query and passage: $s_{lex} \leftarrow \sum_{t \in q \cap p} (w_{q_t} * w_{p_t})$.



Phase 2: Finetuning

• **Multi-Vector Retrieval**. As an extension of dense retrieval, the multi-vector method makes use of the entire output embeddings for the representation of query and passage: $E_q = norm(\mathbf{W}_{mul}^T \mathbf{H_q})$, $E_p = norm(\mathbf{W}_{mul}^T \mathbf{H_p})$, where $\mathbf{W}_{mul} \in \mathbb{R}^{d \times d}$ is the learnable projection matrix. Following ColBert(Khattab and Zaharia, 2020), we use late-interaction to compute the fine-grained relevance score: $s_{mul} \leftarrow \frac{1}{N} \sum_{i=1}^{N} \max_{j=1}^{M} E_q[i] \cdot E_p^T[j]$; N and M are the lengths of query and passage.



The final retrieval result is re-ranked based on the integrated relevance score:

$$s_{rank} \leftarrow w_1 \cdot s_{dense} + w_2 \cdot s_{lex} + w_3 \cdot s_{mul}$$

Loss function

- The weighted sum of different prediction scores:

$$s_{inter} \leftarrow w_1 \cdot s_{dense} + w_2 \cdot s_{lex} + w_3 \cdot s_{mul}.$$

- Contrastive loss for each component:

$$\mathcal{L}_{s(\cdot)} = -\log \frac{\exp(s(q,p^*)/\tau)}{\sum_{p\in\{p^*,P'\}} \exp(s(q,p)/\tau)}.$$

where , p∗ and P′ stand for the positive and negative samples to the query q; s(·) is any of the functions within $s_{dense(.)}, s_{lex(.)}, s_{mul(.)}, s_{inter(.)}$

- The weighted sum of losses:

$$\mathcal{L} \leftarrow \left(\lambda_1 \cdot \mathcal{L}_{dense} + \lambda_2 \cdot \mathcal{L}_{lex} + \lambda_3 \cdot \mathcal{L}_{mul} + \mathcal{L}_{inter}\right)/4.$$

- Hyperparameter setting.

$$w_1 = 1, w_2 = 0.3, w_3 = 1, \lambda_1 = 1, \lambda_2 = 0.1 \text{ and } \lambda_3 = 1$$

Self-knowledge distillation

- Employ the integration score s_{inter} as the teacher, each functional score is a student:

$$\mathcal{L}'_* \leftarrow -p(s_{inter}) * \log p(s_*).$$

where p(·) is the softmax activation; s_* is any of the members within $s_{dense(.)}, s_{lex(.)}, s_{mul(.)}$

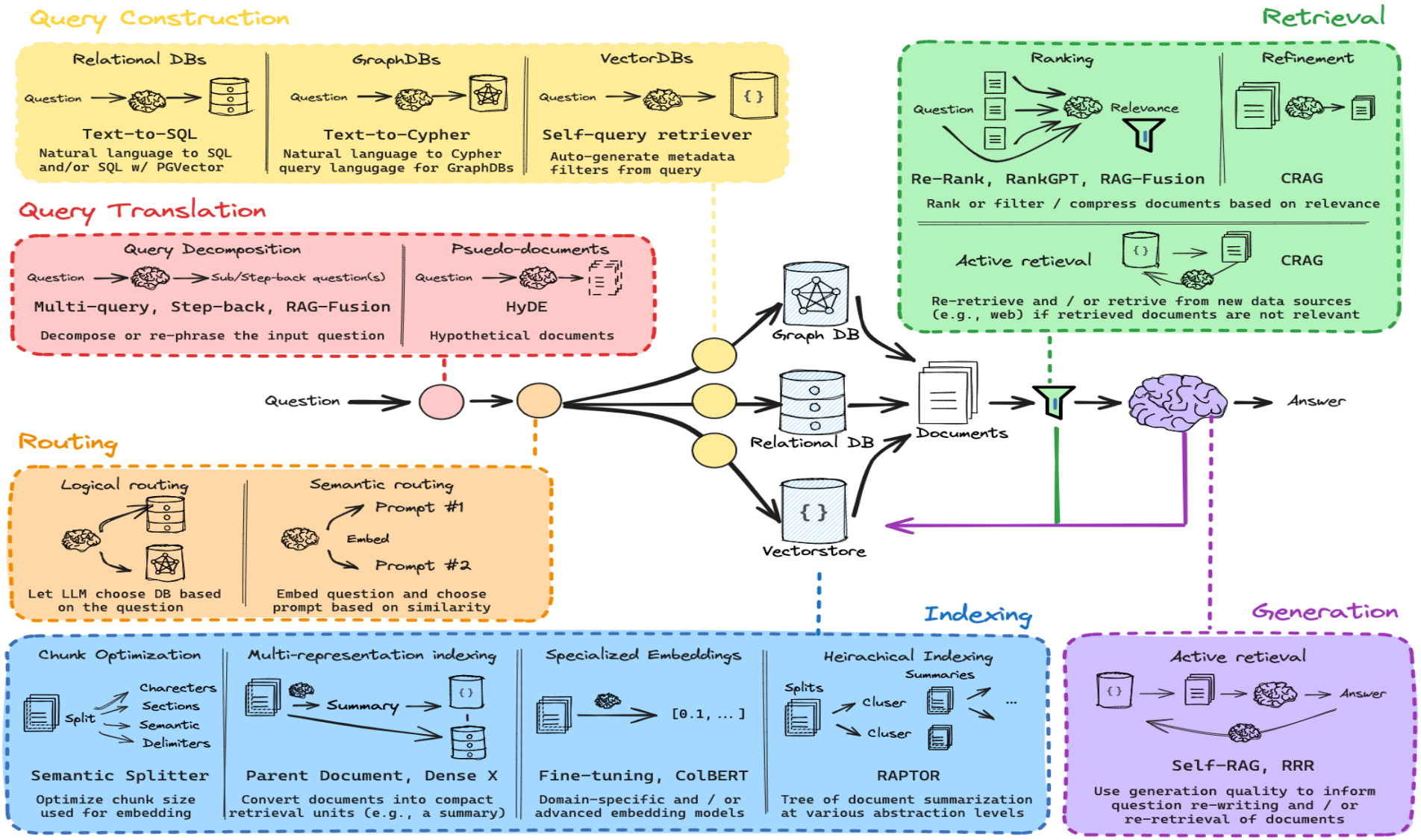
- Integrate and normalize the modified loss function

$$\mathcal{L}' \leftarrow \left(\lambda_1 \cdot \mathcal{L}'_{dense} + \lambda_2 \cdot \mathcal{L}'_{lex} + \lambda_3 \cdot \mathcal{L}'_{mul}\right)/3.$$

- Final loss: The linear combination of contrastive loss and self-knowledge distillation:

$$\mathcal{L}_{final} \leftarrow \left(\mathcal{L} + \mathcal{L}'\right)/2.$$

3. RAG advanced techniques



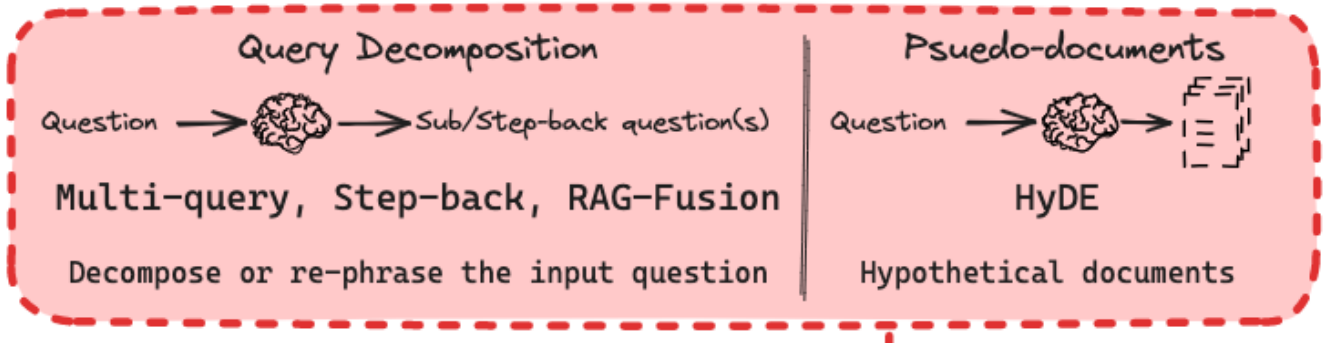
https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_1_to_4.ipynb

Query Translation

- Distance-based retrieval can be sensitive to query wording and imperfect embedding
- Current Approach: Manual prompt tuning ⇒ Tedious

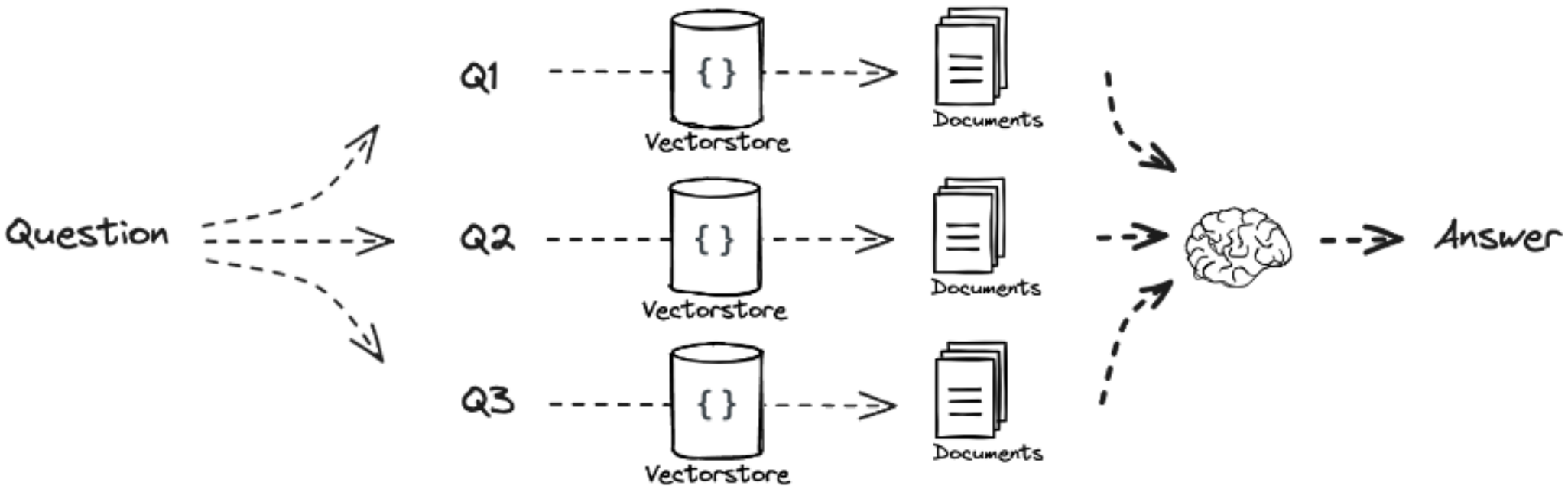
⇒ Automates prompt tuning with LLMs

- Approaches:
  - Multi-query
  - RAG-Fusion
  - Step-back
  - HyDE
  - ...



Multi-Query

Generate different questions from the original one to retrieve more diverse documents



https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_5_to_9.ipynb

Multi-Query

Concatenate retrieved documents into a context, and ask the LLM with the original question

```
# Multi Query: Different Perspectives
template = """You are an AI language model assistant. Your task is to generate five
different versions of the given user question to retrieve relevant documents from a vector
database. By generating multiple perspectives on the user question, your goal is to help
the user overcome some of the limitations of the distance-based similarity search.
Provide these alternative questions separated by newlines. Original question: {question}"""
prompt_perspectives = ChatPromptTemplate.from_template(template)
```

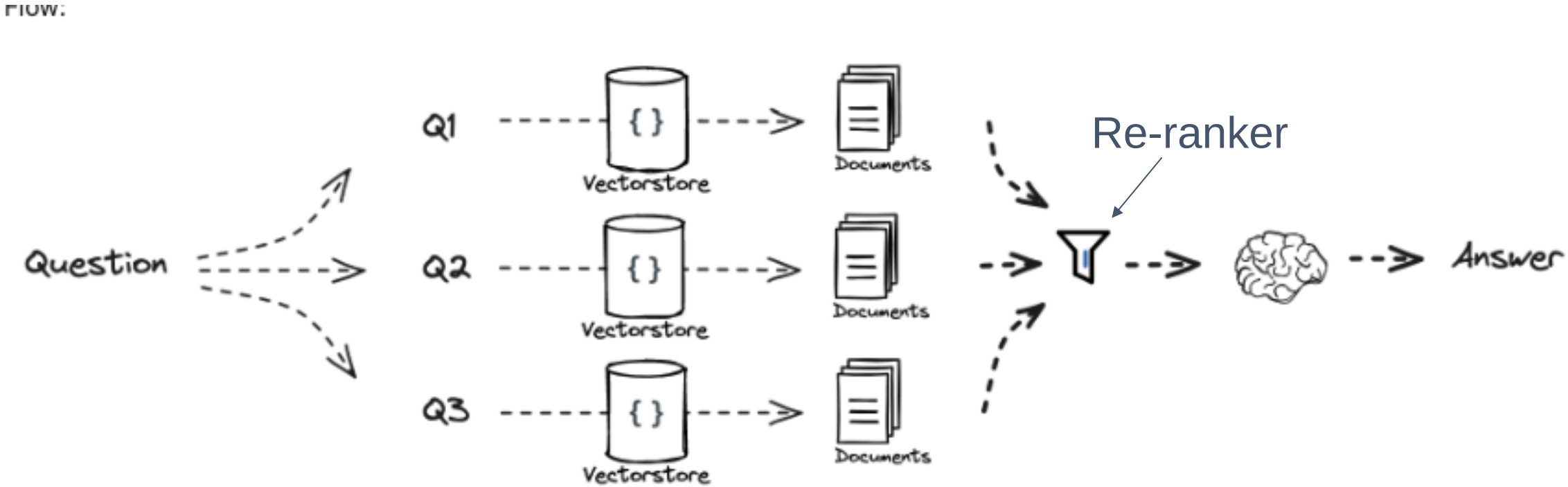
```
# RAG
template = """Answer the following question based on this context:

{context}

Question: {question}
"""
```

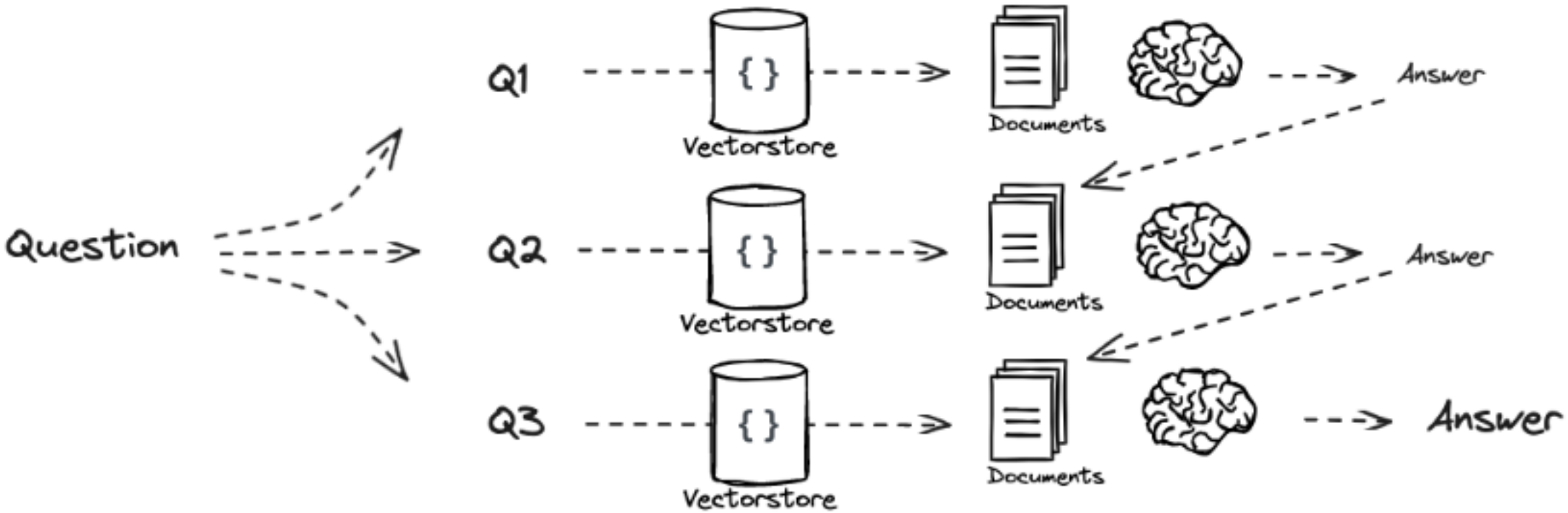
RAG-Fusion

Different from Multi-query, RAG-Fusion add a Re-ranker to put the most relevant documents into the beginning of the context ⇒ Mitigating “Lost-in-the-middle” problem.



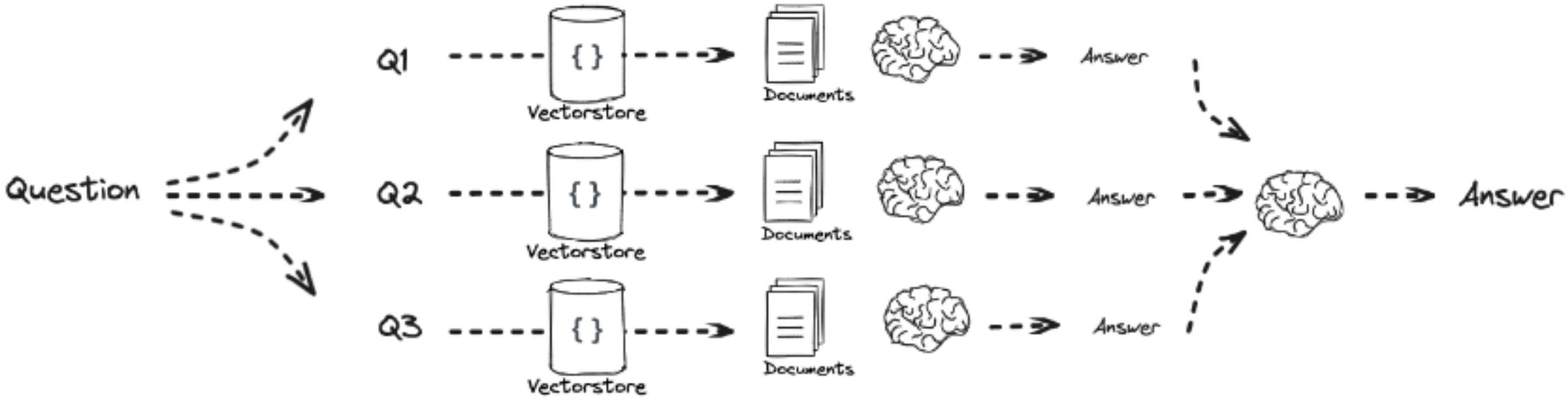
Recursive Query Decomposition

```
# Decomposition
template = """You are a helpful assistant that generates multiple sub-questions related to an input question. \n
The goal is to break down the input into a set of sub-problems / sub-questions that can be answers in isolation.
Generate multiple search queries related to: {question} \n
Output (3 queries):"""
prompt_decomposition = ChatPromptTemplate.from_template(template)
```



https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_5_to_9.ipynb

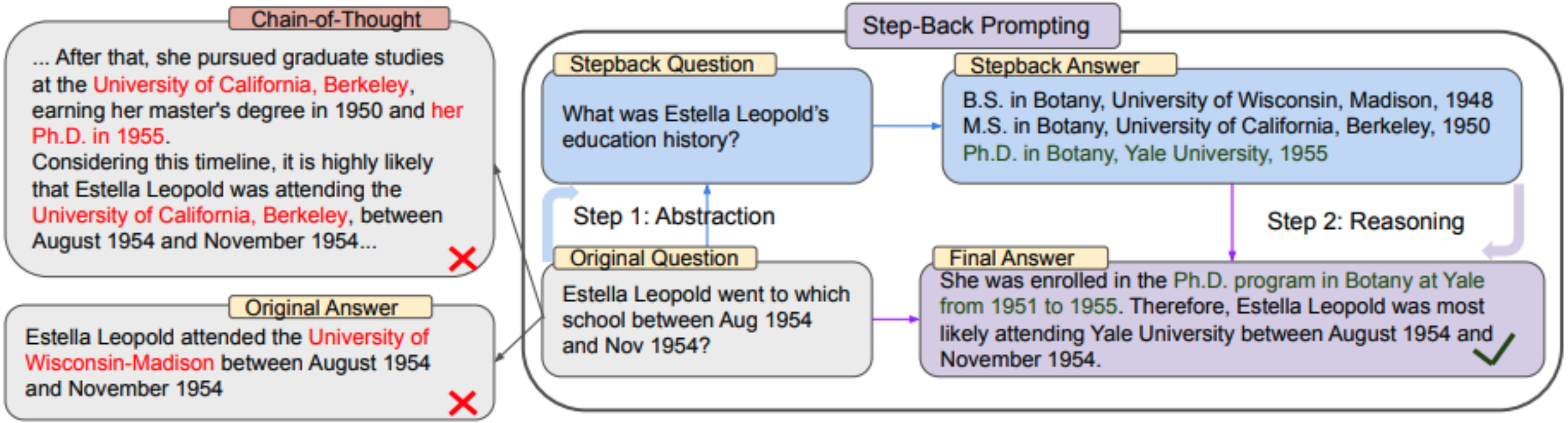
Individual Query Decomposition



Ask sub-questions separately and concatenate all question and answer pairs into context then ask LLM to synthesize the answer for the original question.

Step-Back

Prompt = """You are an expert at world knowledge. Your task is to step back and paraphrase a question to a more generic step-back question, which is easier to answer. Here are a few examples:""",

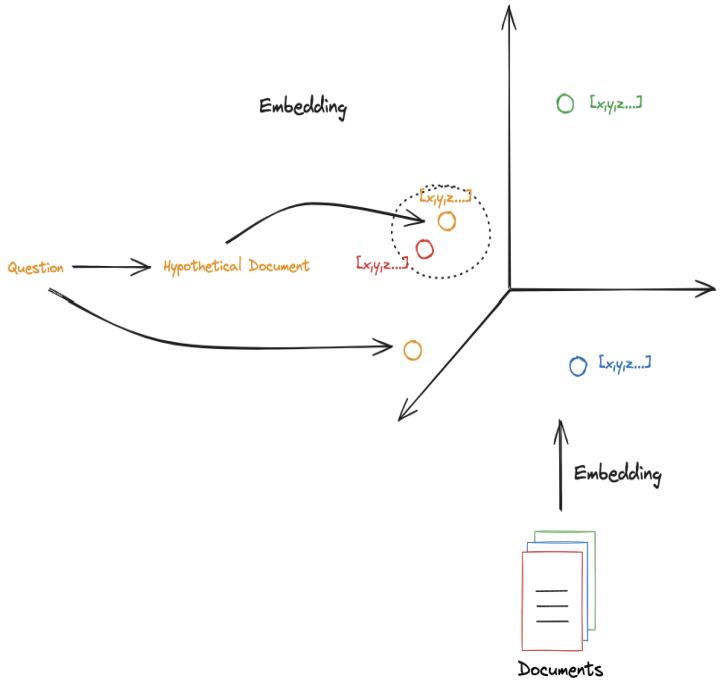


https://arxiv.org/pdf/2310.06117.pdf

HyDE

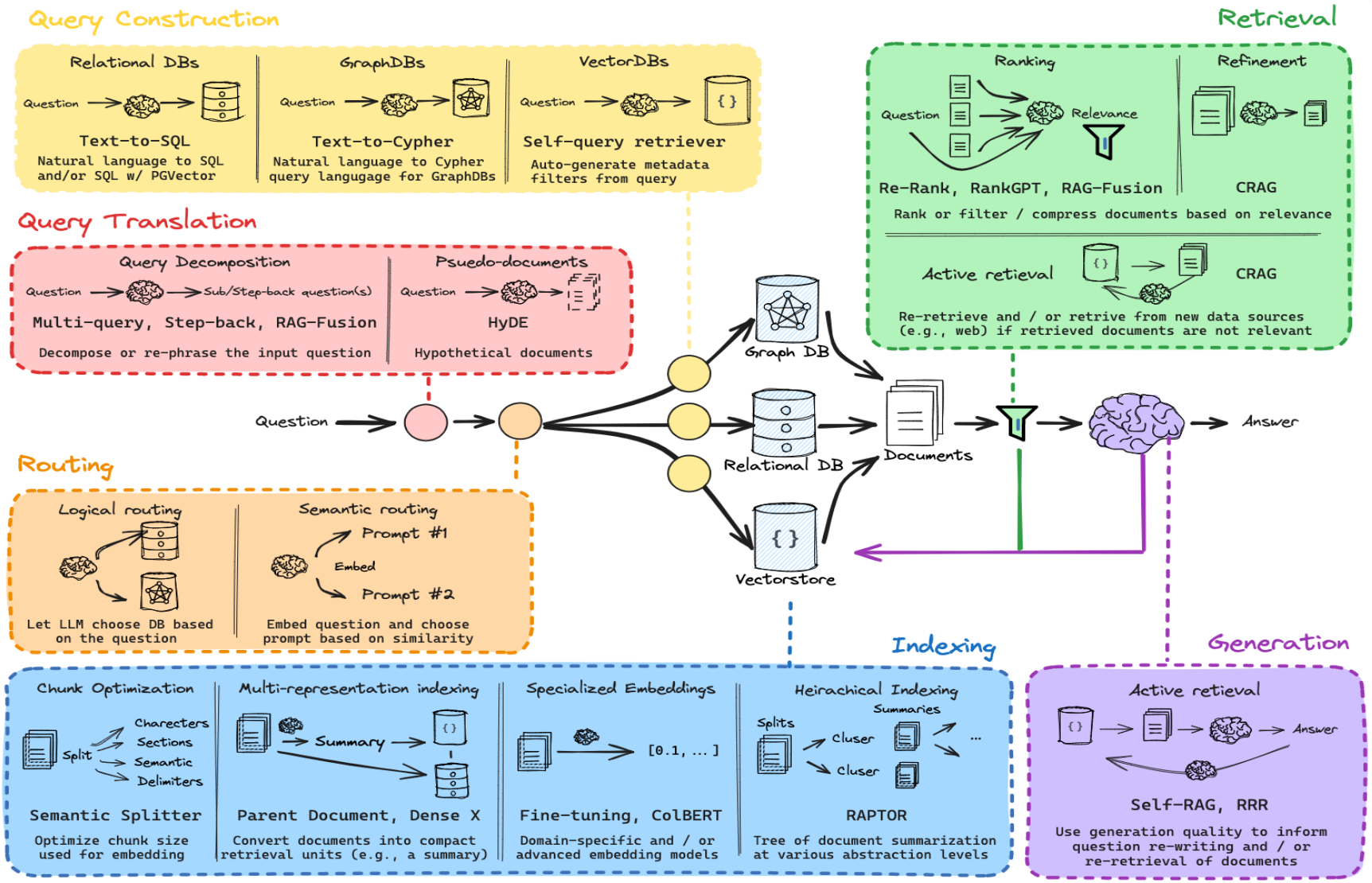
Generates hypothetical answers to queries, embeds them for better retrieval.

```
# HyDE document genration
template = """Please write a scientific paper passage to answer the question
Question: {question}
Passage:"""
prompt_hyde = ChatPromptTemplate.from_template(template)
```

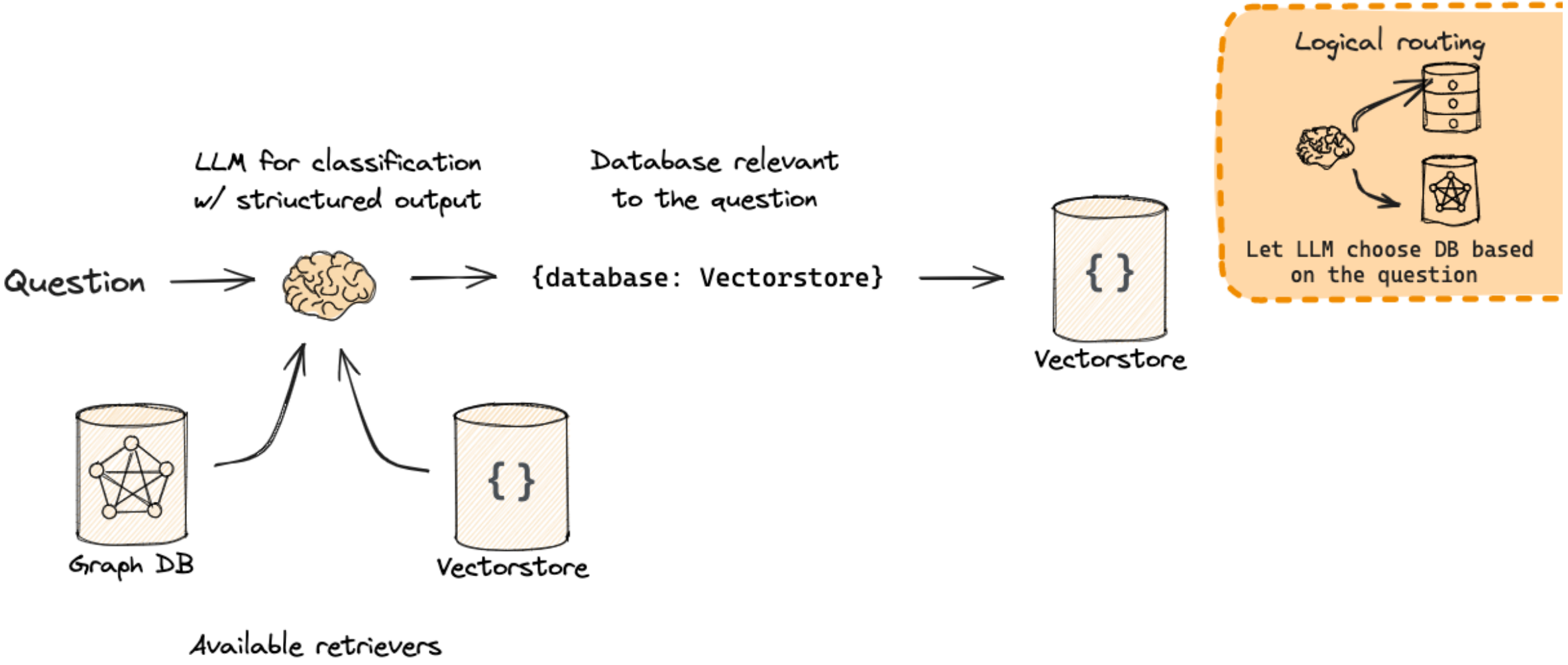


https://github.com/langchain-ai/langchain/blob/master/cookbook/hypothetical_document_embeddings.ipynb

Routing

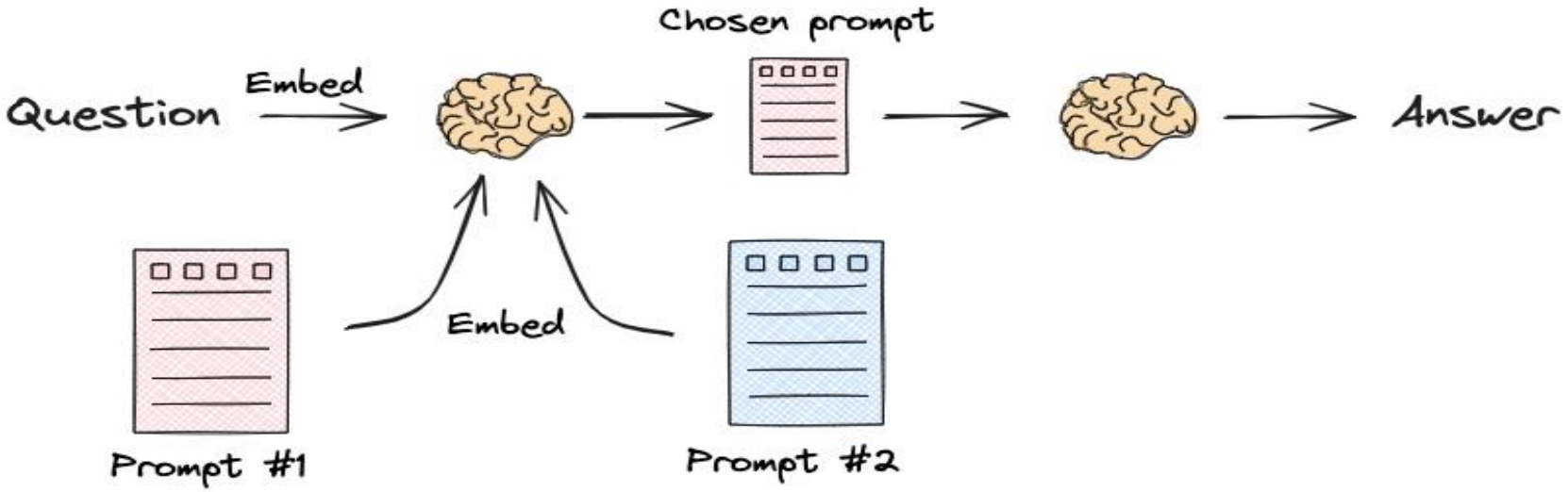

Query Construction
Relational DBs
Question
Text-to-SQL
Natural language to SQL and/or SQL w/ PGVector
GraphDBs
Question
Text-to-Cypher
Natural language to Cypher query langugage for GraphDBs
VectorDBs
Question
{ }
Self-query retriever
Auto-generate metadata filters from query
Retrieval
Ranking
Question
Relevance
Refinement
Re-Rank, RankGPT, RAG-Fusion
CRAG
Rank or filter / compress documents based on relevance
Active retieval
{ }
CRAG
Re-retrieve and / or retrive from new data sources (e.g., web) if retrieved documents are not relevant
Query Translation
Query Decomposition
Question
Sub/Step-back question(s)
Multi-query, Step-back, RAG-Fusion
Decompose or re-phrase the input question
Psuedo-documents
Question
HyDE
Hypothetical documents
Graph DB
Question
Relational DB
Documents
Answer
Vectorstore
{ }
Routing
Logical routing
Let LLM choose DB based on the question
Semantic routing
Prompt #1
Embed
Prompt #2
Embed question and choose prompt based on similarity
Indexing
Generation
Chunk Optimization
Split
Charecters
Sections
Semantic
Delimiters
Semantic Splitter
Optimize chunk size used for embedding
Multi-representation indexing
Summary
{ }
Parent Document, Dense X
Convert documents into compact retrieval units (e.g., a summary)
Specialized Embeddings
[0.1, ...]
Fine-tuning, ColBERT
Domain-specific and / or advanced embedding models
Heirachical Indexing
Splits
Cluser
Cluser
Summaries
...
RAPTOR
Tree of document summarization at various abstraction levels
Active retieval
{ }
Answer
Self-RAG, RRR
Use generation quality to inform question re-writing and / or re-retrieval of documents

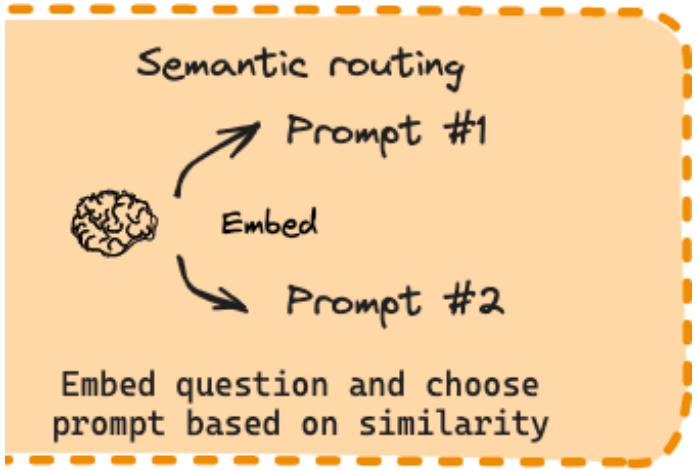
Routing



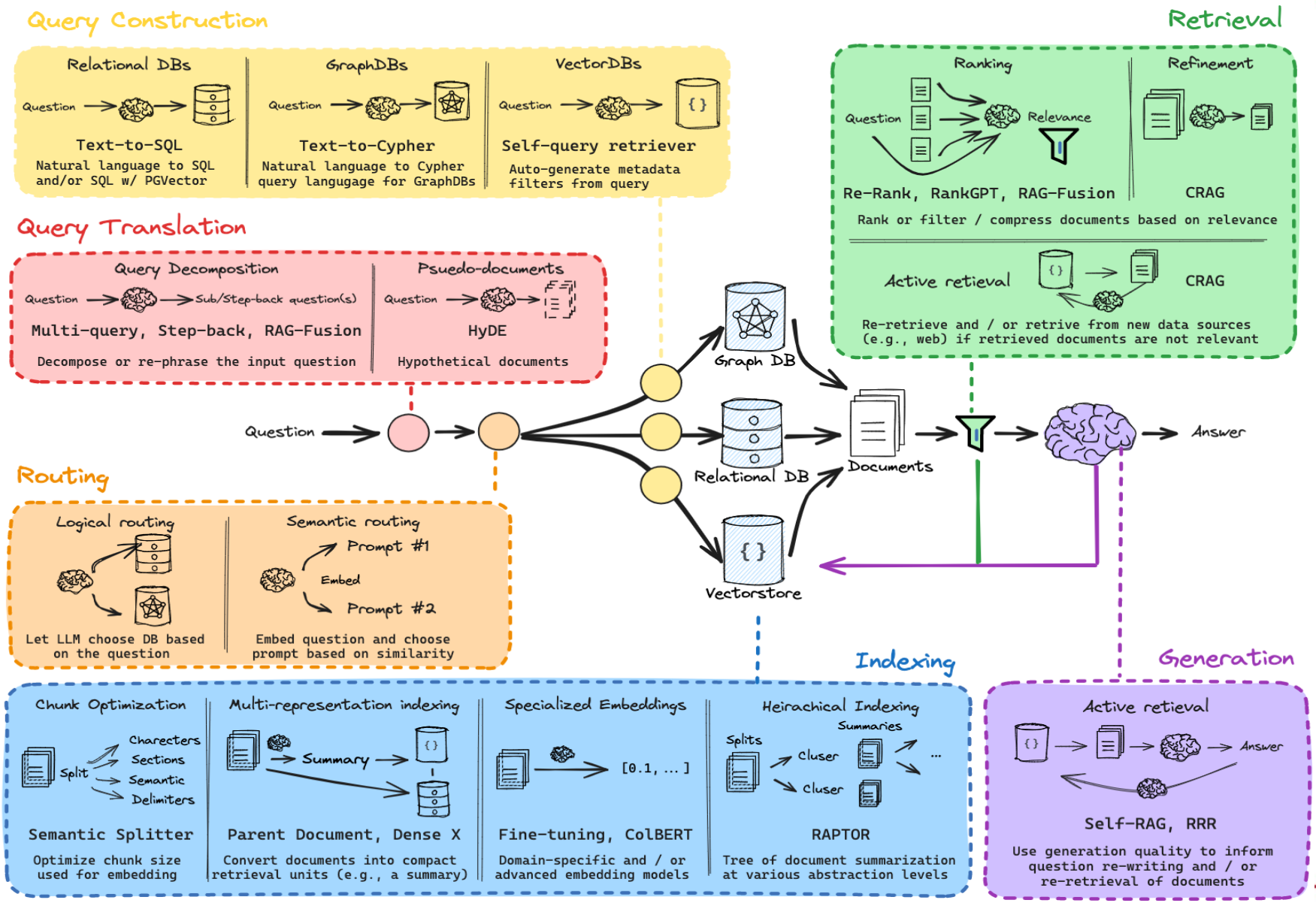
https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_10_and_11.ipynb

Routing





https://github.com/langchain-ai/rag-from-scratch/blob/main/rag_from_scratch_10_and_11.ipynb

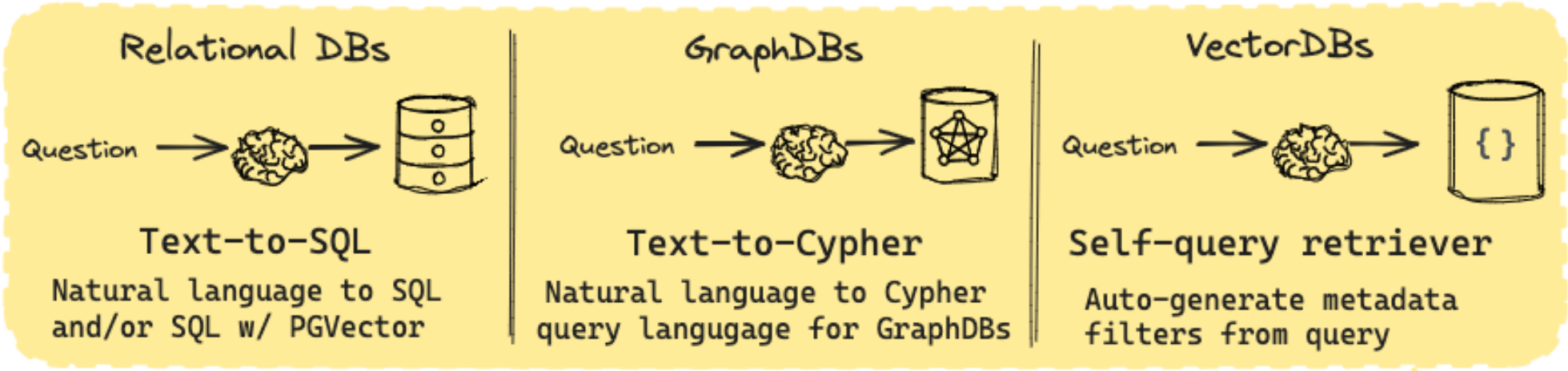

Query Construction
Relational DBs
Question
Text-to-SQL
Natural language to SQL and/or SQL w/ PGVector
GraphDBs
Question
Text-to-Cypher
Natural language to Cypher query langugage for GraphDBs
VectorDBs
Question
{ }
Self-query retriever
Auto-generate metadata filters from query
Retrieval
Ranking
Question
Relevance
Re-Rank, RankGPT, RAG-Fusion
Refinement
CRAG
Rank or filter / compress documents based on relevance
Active retieval
{ }
CRAG
Re-retrieve and / or retrive from new data sources (e.g., web) if retrieved documents are not relevant
Query Translation
Query Decomposition
Question
Sub/Step-back question(s)
Multi-query, Step-back, RAG-Fusion
Decompose or re-phrase the input question
Psuedo-documents
Question
HyDE
Hypothetical documents
Graph DB
Question
Relational DB
Documents
Answer
Vectorstore
{ }
Routing
Logical routing
Let LLM choose DB based on the question
Semantic routing
Prompt #1
Embed
Prompt #2
Embed question and choose prompt based on similarity
Indexing
Generation
Chunk Optimization
Split
Charecters
Sections
Semantic
Delimiters
Semantic Splitter
Optimize chunk size used for embedding
Multi-representation indexing
Summary
{ }
Parent Document, Dense X
Convert documents into compact retrieval units (e.g., a summary)
Specialized Embeddings
[0.1, ...]
Fine-tuning, ColBERT
Domain-specific and / or advanced embedding models
Heirachical Indexing
Splits
Summaries
Cluser
Cluser
...
RAPTOR
Tree of document summarization at various abstraction levels
Active retieval
{ }
Answer
Self-RAG, RRR
Use generation quality to inform question re-writing and / or re-retrieval of documents

Query Construction

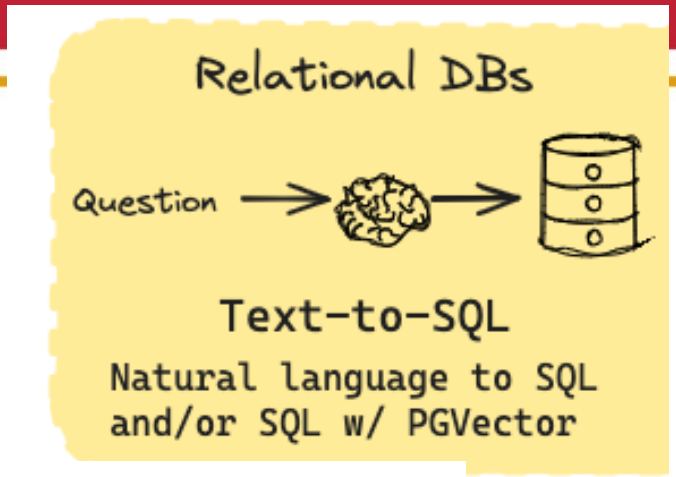
- Most data has some structure: SQL, Graph
- Previous use embedding to retrieve unstructured data

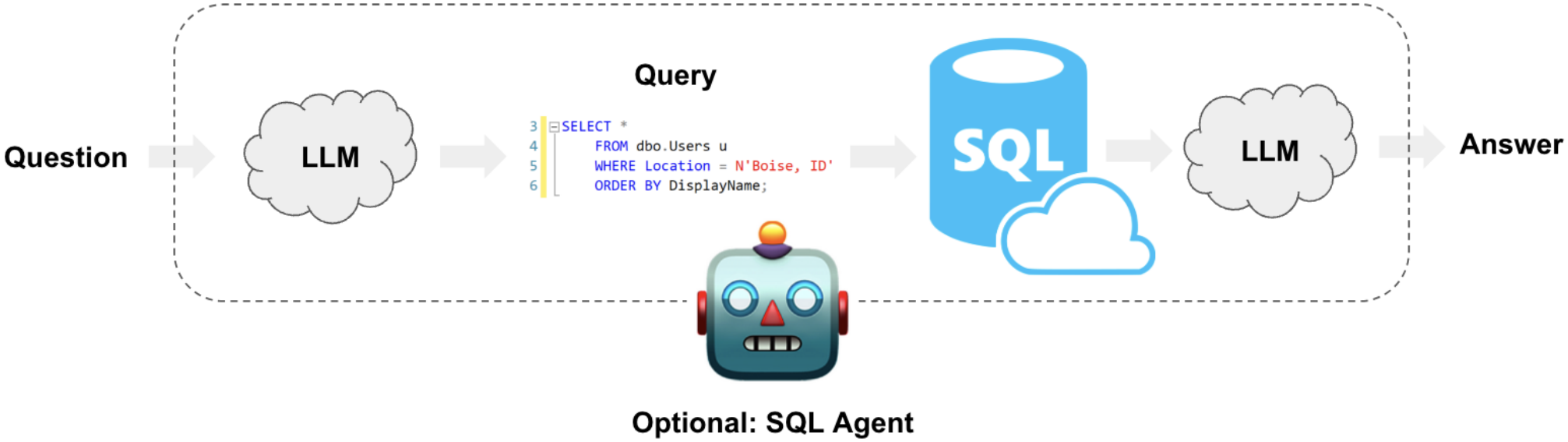
⇒ How's about structured data?

- Query Construction converts natural language into a specific query syntax

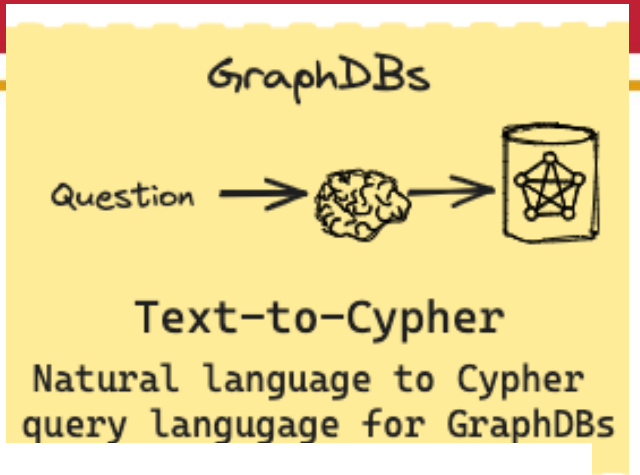


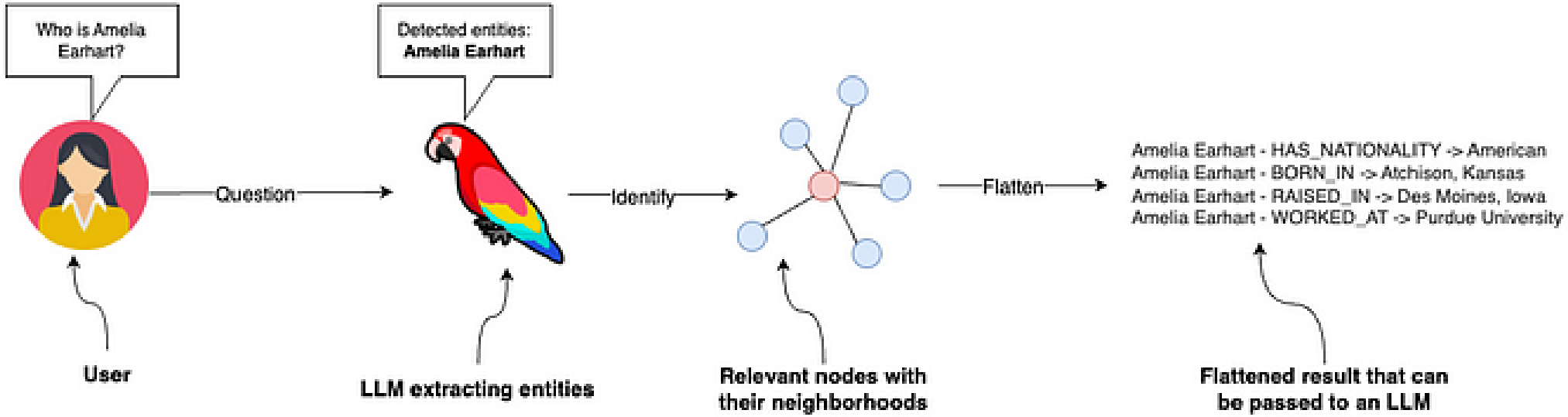
Query Construction

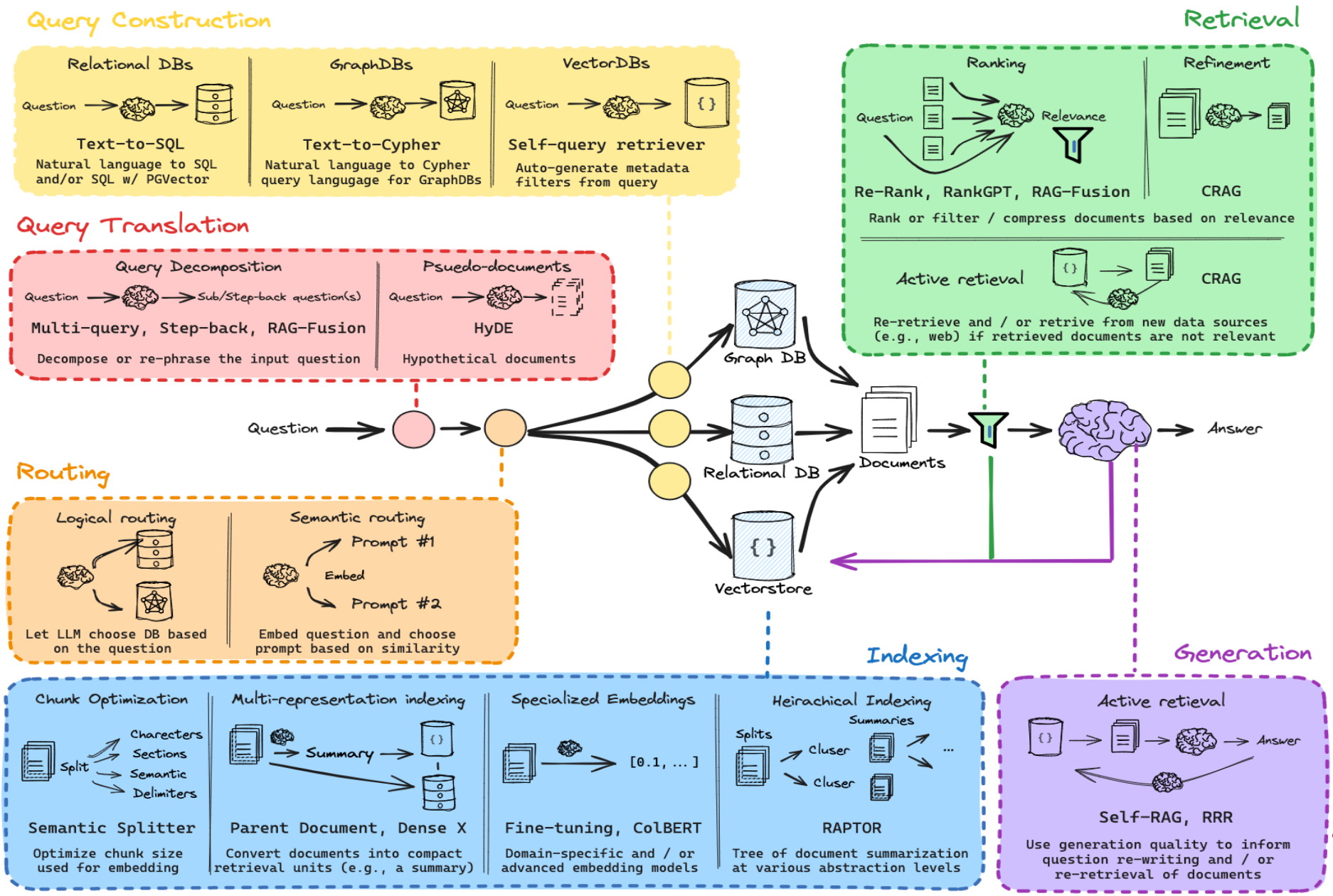




Query Construction

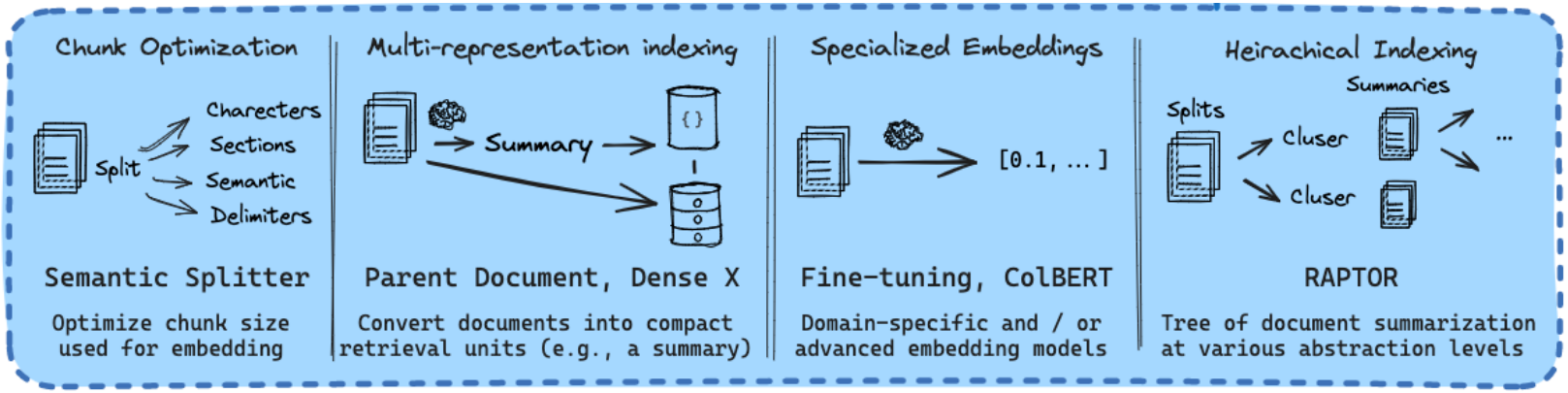


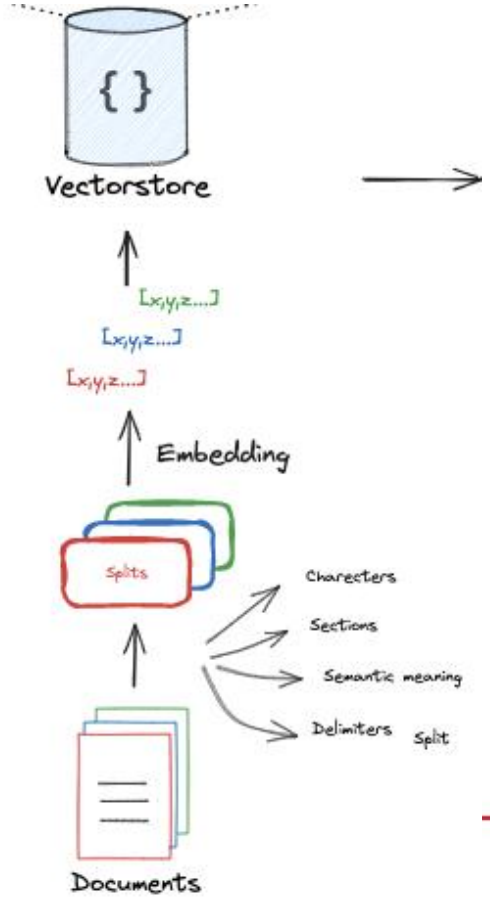



Query Construction
Relational DBs
Question
Text-to-SQL
Natural language to SQL and/or SQL w/ PGVector
GraphDBs
Question
Text-to-Cypher
Natural language to Cypher query langugage for GraphDBs
VectorDBs
Question
{ }
Self-query retriever
Auto-generate metadata filters from query
Retrieval
Ranking
Question
Relevance
Re-Rank, RankGPT, RAG-Fusion
Refinement
CRAG
Rank or filter / compress documents based on relevance
Active retieval
{ }
CRAG
Re-retrieve and / or retrive from new data sources (e.g., web) if retrieved documents are not relevant
Query Translation
Query Decomposition
Question
Sub/Step-back question(s)
Multi-query, Step-back, RAG-Fusion
Decompose or re-phrase the input question
Psuedo-documents
Question
HyDE
Hypothetical documents
Graph DB
Question
Relational DB
Documents
Answer
Vectorstore
{ }
Routing
Logical routing
Let LLM choose DB based on the question
Semantic routing
Prompt #1
Embed
Prompt #2
Embed question and choose prompt based on similarity
Indexing
Chunk Optimization
Split
Charecters
Sections
Semantic
Delimiters
Semantic Splitter
Optimize chunk size used for embedding
Multi-representation indexing
Summary
{ }
Parent Document, Dense X
Convert documents into compact retrieval units (e.g., a summary)
Specialized Embeddings
[0.1, ...]
Fine-tuning, ColBERT
Domain-specific and / or advanced embedding models
Heirachical Indexing
Splits
Summaries
Cluser
Cluser
...
RAPTOR
Tree of document summarization at various abstraction levels
Generation
Active retieval
{ }
Answer
Self-RAG, RRR
Use generation quality to inform question re-writing and / or re-retrieval of documents

Indexing

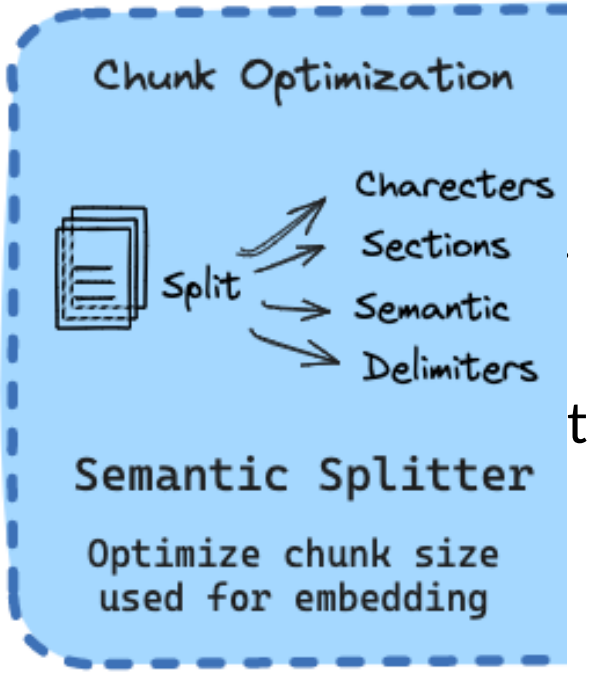
- Documents will be processed, segmented, transformed into Embeddings.
- The quality of index construction determines whether the correct context can be obtained in the retrieval phase.





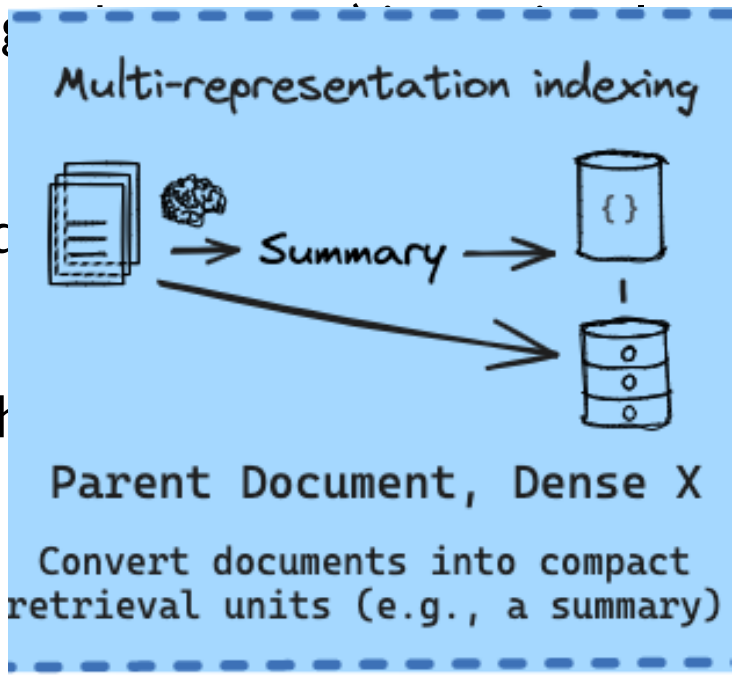
Indexing

- **Balancing Context and Efficiency:**
  - **Fixed-Size Chunking:**
    - Common method (e.g., 100, 256, 512 tokens).
    - Larger chunks: More context, but more noise (longer p
    - Smaller chunks: Less noise, but less context.
  - **Alternative Approaches:**
    - Recursive Splits & Sliding Windows: Layered retrieval t complex.
- **Challenge:** Finding the optimal balance between semantic compl length.



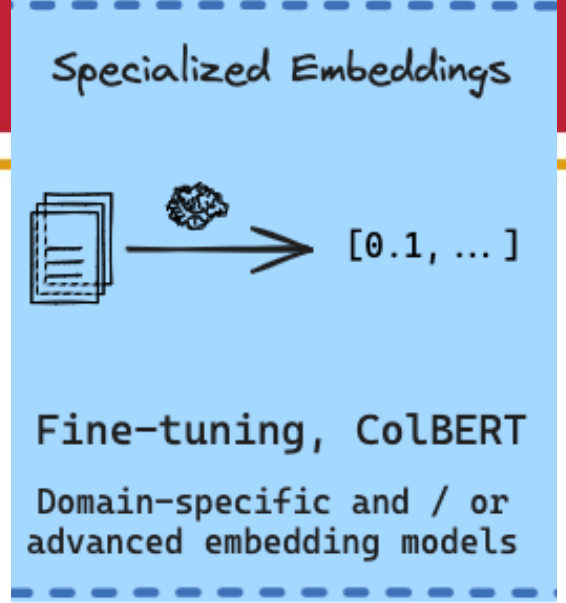
Multi-Representation Indexing

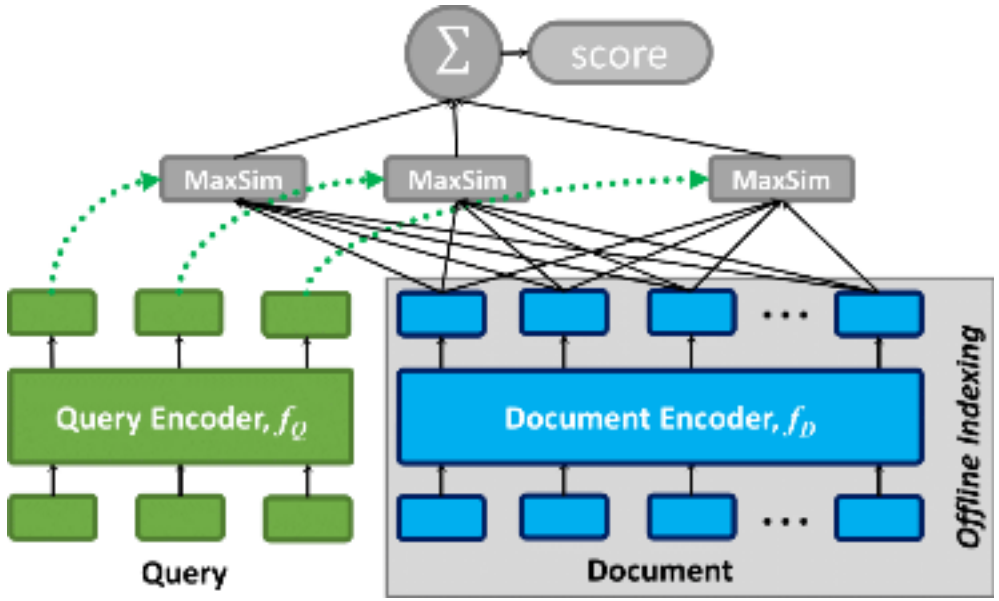
- **Challenge:** Balancing meaning (small chunks) and context (larg
- **Parent Document Retriever:**
  - Splits documents into small chunks for accurate embedd
  - Stores parent document IDs for each chunk.
  - Retrieves relevant chunks during search.
  - Returns the complete parent documents for retrieved ch

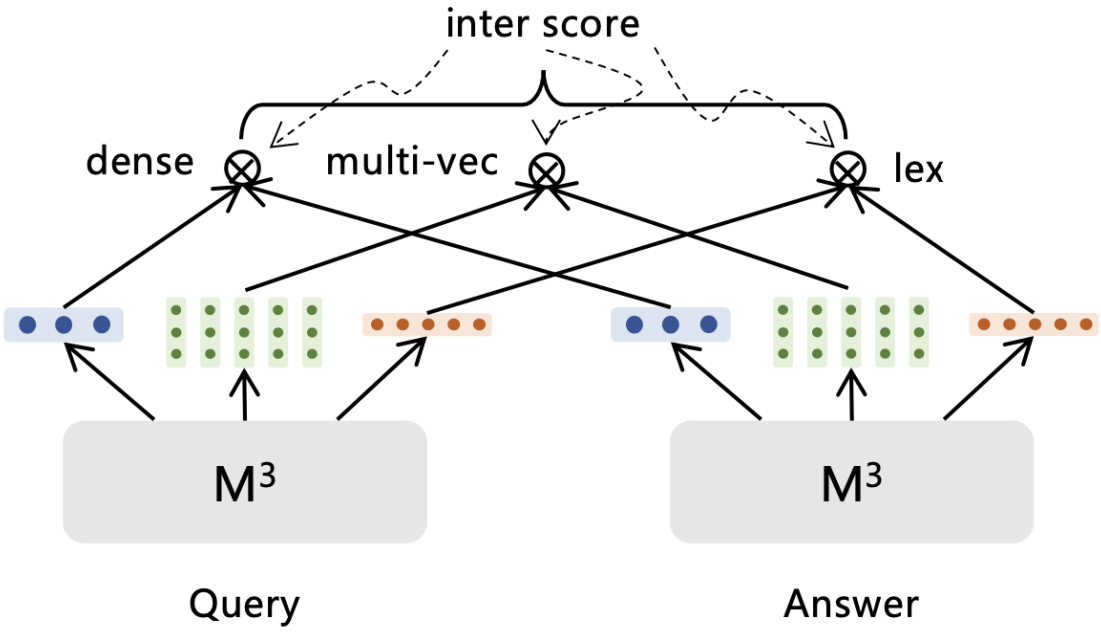


Specialized Embeddings

- Improve embedding leads to better retrieval
- Domain-specific Finetuning embedding models
- Advanced embeddings models



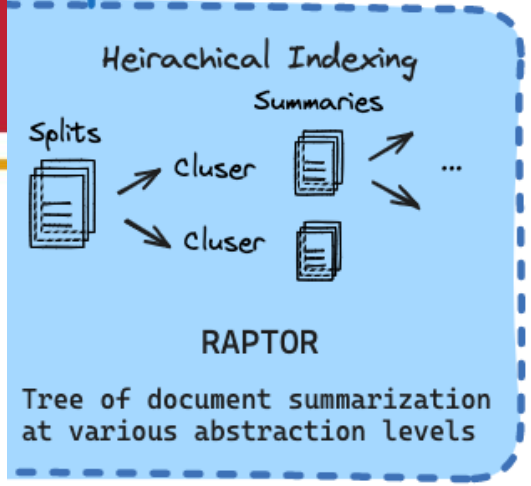


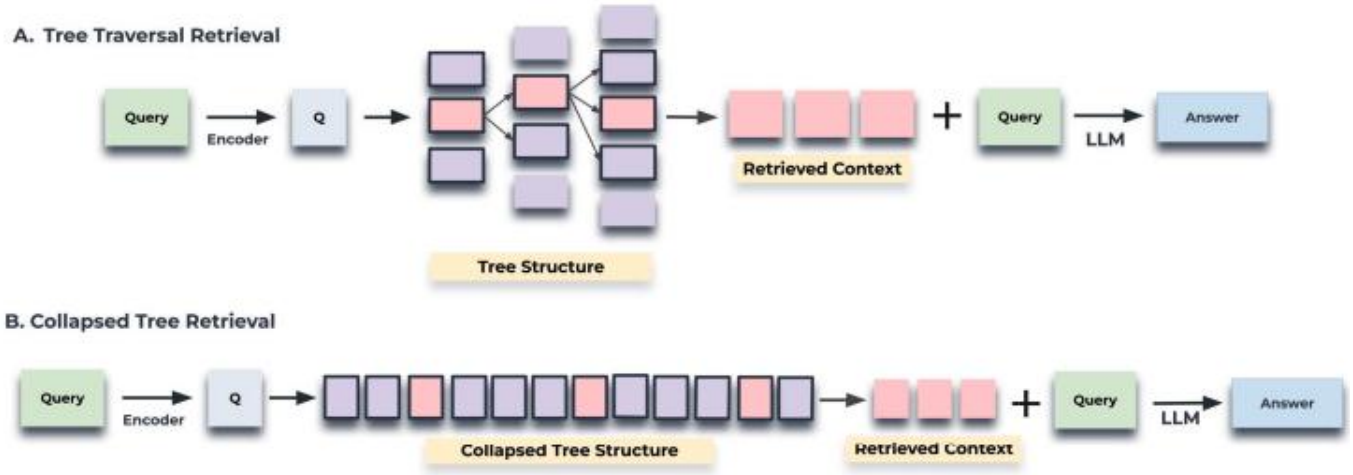


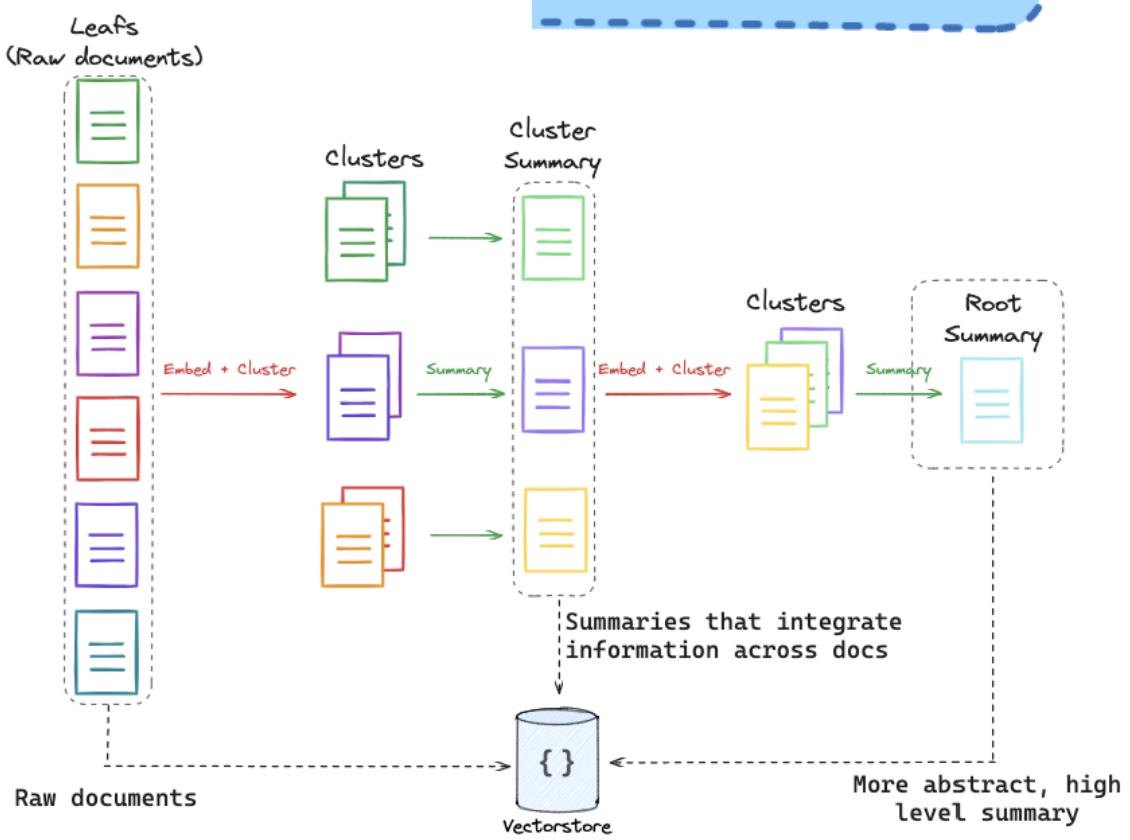
ColBERT: https://github.com/stanford-futuredata/ColBERT

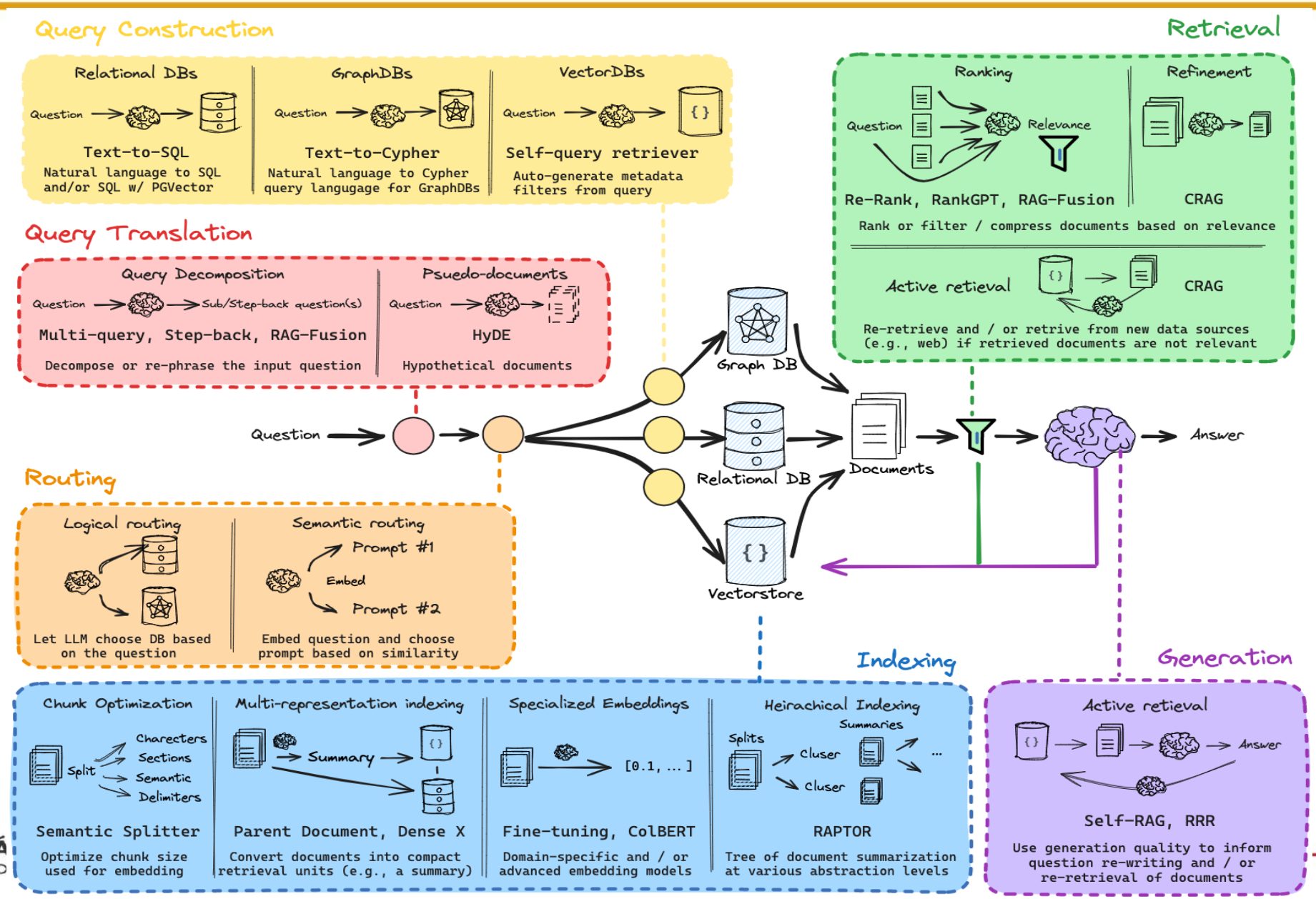
bge-m3 embedding: https://huggingface.co/BAAI/bge-m3

Hierarchical Indexing







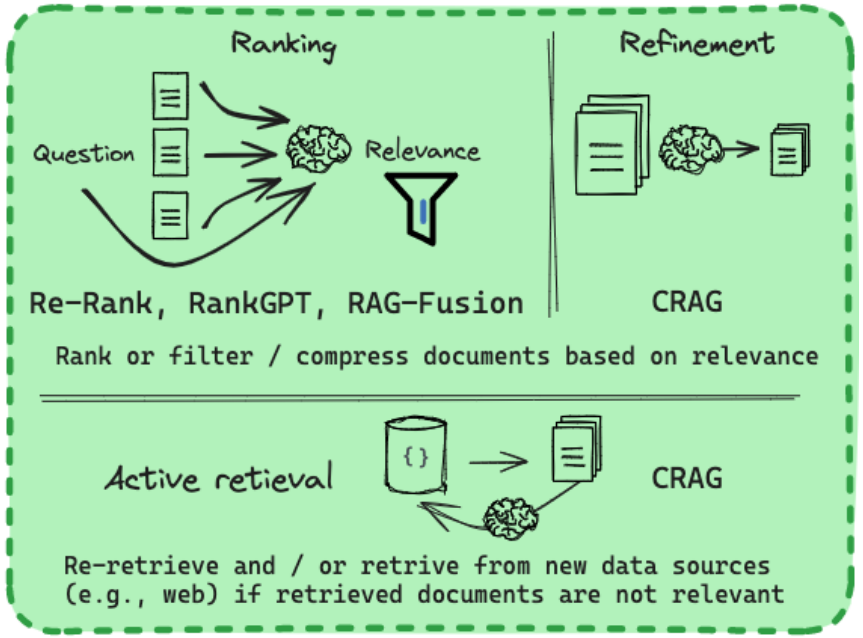

Query Construction
Relational DBs
Question
Text-to-SQL
Natural language to SQL and/or SQL w/ PGVector
GraphDBs
Question
Text-to-Cypher
Natural language to Cypher query langugage for GraphDBs
VectorDBs
Question
{ }
Self-query retriever
Auto-generate metadata filters from query
Retrieval
Ranking
Question
Relevance
Refinement
Re-Rank, RankGPT, RAG-Fusion
CRAG
Rank or filter / compress documents based on relevance
Active retieval
{ }
CRAG
Re-retrieve and / or retrive from new data sources (e.g., web) if retrieved documents are not relevant
Query Translation
Query Decomposition
Question
Sub/Step-back question(s)
Multi-query, Step-back, RAG-Fusion
Decompose or re-phrase the input question
Psuedo-documents
Question
HyDE
Hypothetical documents
Graph DB
Question
Relational DB
Documents
Answer
Vectorstore
{ }
Routing
Logical routing
Let LLM choose DB based on the question
Semantic routing
Prompt #1
Embed
Prompt #2
Embed question and choose prompt based on similarity
Indexing
Generation
Chunk Optimization
Split
Charecters
Sections
Semantic
Delimiters
Semantic Splitter
Optimize chunk size used for embedding
Multi-representation indexing
Summary
{ }
Parent Document, Dense X
Convert documents into compact retrieval units (e.g., a summary)
Specialized Embeddings
[0.1, ...]
Fine-tuning, ColBERT
Domain-specific and / or advanced embedding models
Heirachical Indexing
Splits
Summaries
Cluser
Cluser
...
RAPTOR
Tree of document summarization at various abstraction levels
Active retieval
{ }
Answer
Self-RAG, RRR
Use generation quality to inform question re-writing and / or re-retrieval of documents

Retrieval

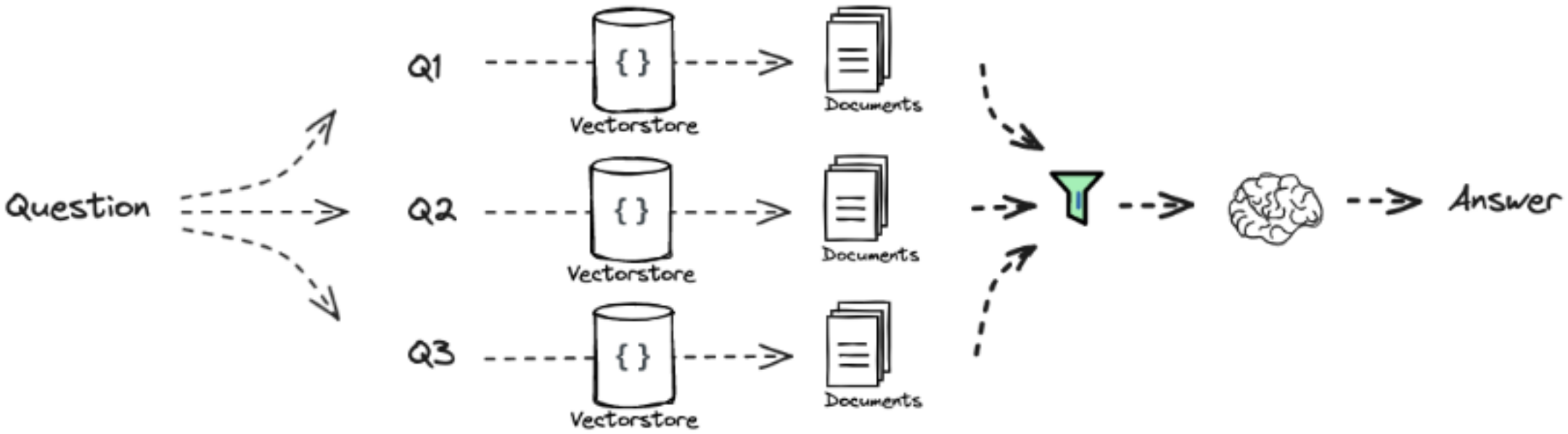
- RAG relies on external knowledge to enhanc

⇒ The type of **retrieval source** and the **granula** both affect the final generation results

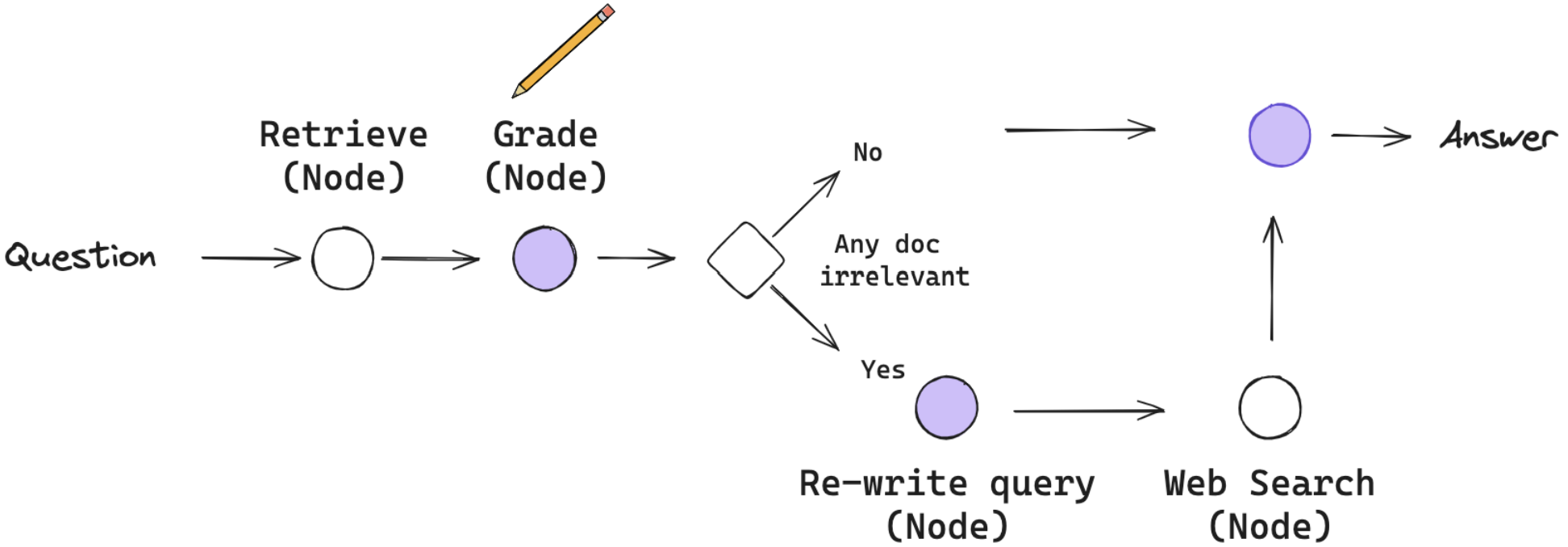
- So far, we mentioned unstructured/ structur generated content
- In text, retrieval granularity ranges from fine Token, Phrase, Sentence, Proposition, Chunl



RAG-Fusion


Question
Q1
Q2
Q3
Vectorstore
Vectorstore
Vectorstore
Documents
Documents
Documents
Answer

Corrective-RAG



https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_crag.ipynb

Grade node's prompt to check whether the given document is relevant or not

```
# Prompt
system = """You are a grader assessing relevance of a retrieved document to a user question. \n
    If the document contains keyword(s) or semantic meaning related to the question, grade it as relevant. \n
    Give a binary score 'yes' or 'no' score to indicate whether the document is relevant to the question."""
grade_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", "Retrieved document: \n\n {document} \n\n User question: {question}"),
    ]
)
```

If does not find any relevant docs based on the input question, ask LLM to re-write the question then use this question for web search

```
question = "agent memory"
```

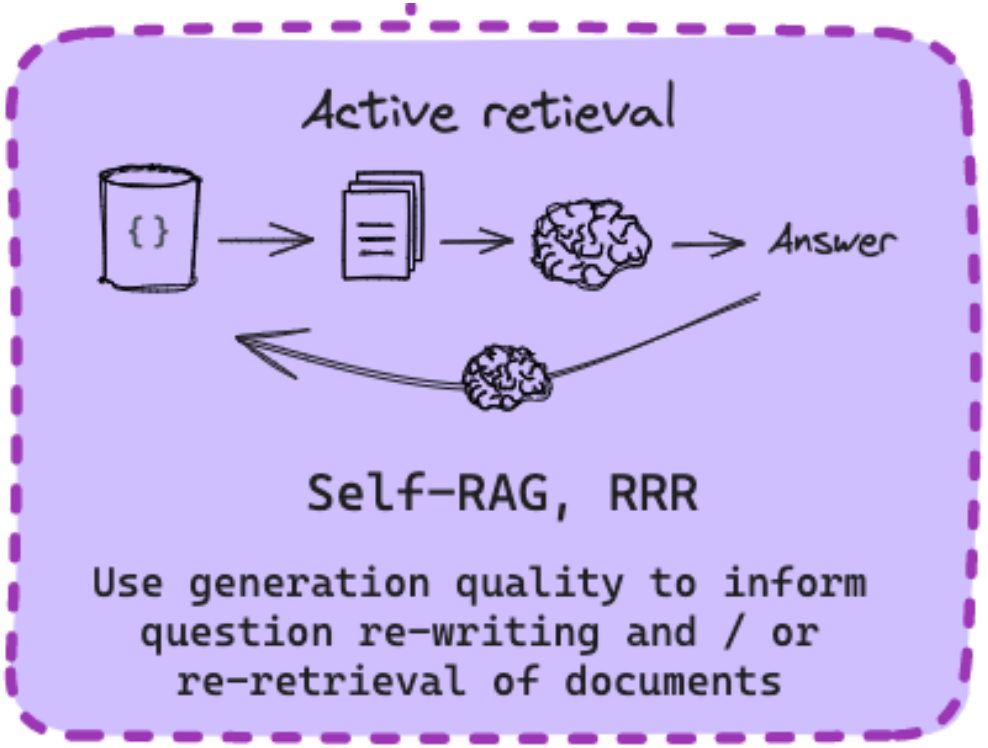
```
# Prompt
system = """You a question re-writer that converts an input question to a better version that is optimized \n
     for web search. Look at the input and try to reason about the underlying semantic intent / meaning."""
re_write_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        (
            "human",
            "Here is the initial question: \n\n {question} \n Formulate an improved question.",
        ),
    ]
)

question_rewriter = re_write_prompt | llm | StrOutputParser()
question_rewriter.invoke({"question": question})
```

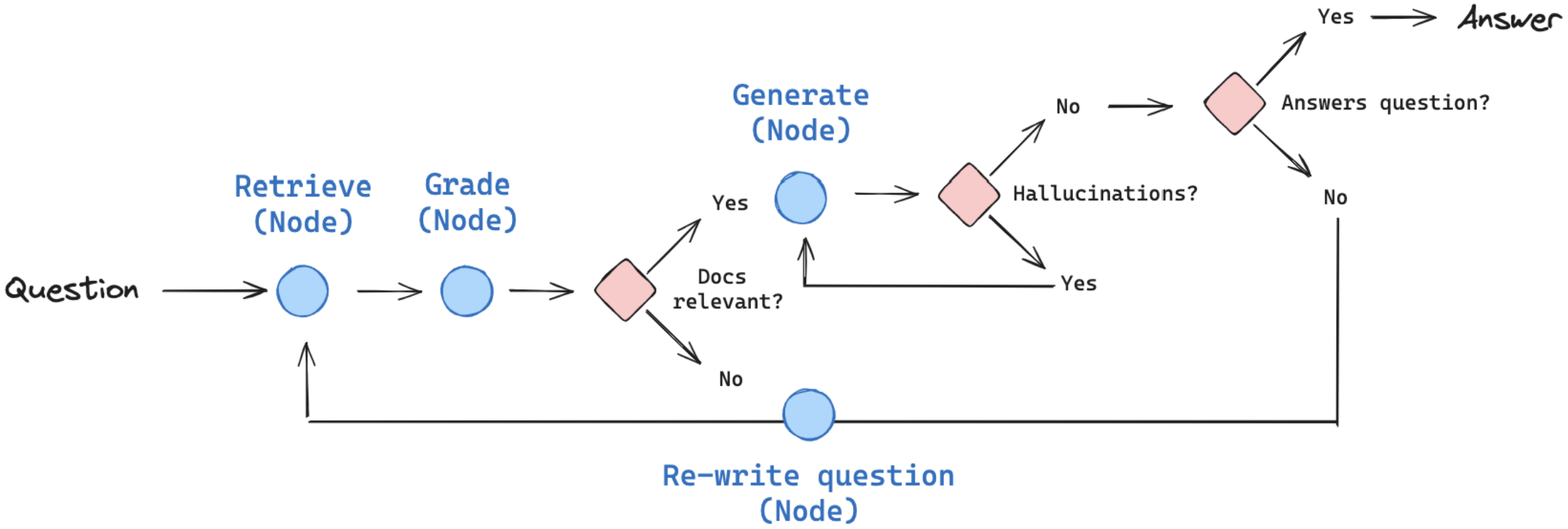
```
'What is the role of memory in artificial intelligence agents?'
```

Generation

- After retrieval, it is not good to directly to LLM
- Problem:
  - Redundant infor interfere with final gene
  - “Lost in the middle” problem with long cc
- Solutions
  - Reranking: ColBert, Reranker models, etc
  - Context compression: uses small LM to r



Self-RAG



https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_self_rag.ipynb

Thực hành